



School of Computing, Engineering and Digital Technologies

Department of Computing and Games

Teesside University

Middlesbrough TS1 3BA

Setting Up Firewall and Monitoring System for a Web Server

*An analysis, design and implementation report for the development of
firewall and monitoring system for a web server.*

Module Name: **Computing Masters Project**

Module code:

Date: 03/09/2024

Submitted in partial requirements for the degree of ***MSc Cybersecurity***

ANUGRAH KIZHAKKE VEEDU

Student ID:

Word count: 8693

Supervisor:

Acknowledgement

My honest appreciation goes to my supervisor, Mr. Harry Stewart, for his guidance and support throughout the dissertation. And I would like to thank my friends and family who supported me through my tough times and gave emotional support. I thank God for helping me through my rough situations.

Setting Up Firewall and Monitoring System for a web server

An analysis, design and implementation report for the development of firewall and monitoring system for a web server.

Abstract

This thesis provides a thorough method of improving network security through the integration of sophisticated monitoring tools, an intrusion detection system (IDS), and a strong firewall into one, integrated framework. Protecting critical infrastructures requires the implementation of multi-layered security solutions due to the increasing complexity and frequency of cyber threats. It focuses on creating and putting together a network security solution that protects a web application by utilizing open-source technology. The system's affordability, scalability and ability to deliver real-time insights into network activity allow for proactive threat identification and response. This research project advances the field of network security by providing a workable, open-source solution suitable for higher education or small-to medium-sized companies. Future enhancements, such as adding more monitoring tools or extending the firewall's capacity for handling bigger networks, are made possible by the system's flexibility. In addition to addressing present security issues, this thesis provides the groundwork for future developments in network defensive strategies.

TABLE OF CONTENTS

Acknowledgement	1
Abstract.....	2
List of Tables	5
List of Figures.....	5
List of Abbreviations	6
CHAPTER 1	7
1. Introduction.....	7
1.1 Background of research	7
1.2 Research Questions	8
1.3 Problem of statement	9
1.4 Scope of study.....	9
1.5 Research Objectives.....	9
1.6 Thesis Structure.....	10
CHAPTER 2	11
2. Literature Review	11
2.1 Historical development of Network Security.....	11
2.2 Evolution of Firewall Technologies	12
2.2.1 Different types of Firewalls	13
2.3 Monitoring System.....	16
2.3.1 Advances in Monitoring Systems	16
2.3.2 Types of monitoring Systems	17
2.4 Intrusion Detection Systems	19
2.4.1 Importance of Intrusion Detection Systems	20
2.4.2 Advances in Intrusion Detection Systems	20
2.5 Related Previous Research Paper.....	21
CHAPTER 3	23
3. Methodology.....	23
3.1 Research Design.....	23
3.2 Tools used.....	23
3.3 Network Topology Map	24
3.4 Integration process	26
3.5 Data processing method.....	27
CHAPTER 4	28
4. Implementation	28
4.1 Setting up Environment	28

4.2 Setting Pfsense	29
4.3 Setting Ubuntu server.....	30
4.4 Setting Apache2	32
4.5 Setting ELK stack	33
4.6 Setting Suricata and Filebeat.....	36
4.7. Setting Pfsense Rules	38
4.8 Setting Router rules.....	39
4.9 Setting DNS	41
CHAPTER 5	41
5. Accuracy and Testing.....	41
5.1 Web application monitoring.....	41
5.1.1 Web service status.....	41
5.1.2 Collecting web logs	43
5.2 Network monitoring.....	44
5.2.1 Network service status.....	44
5.2.2 Collecting network logs.....	45
5.3 Simulation Result.....	46
CHAPTER 6	48
6. Conclusion, Limitations and Future Works	48
6.1 Conclusion	48
6.2 Limitations	48
6.3 Future Works.....	49
References.....	51
Appendices	55
Appendix A: Commands used for Ubuntu server	55
Appendix B: Installation of Apache2.....	55
Appendix C: Commands, HTML and styles used of web application.....	55
Appendix D: Deploying website and configuration	61
Appendix E: UFW in ubuntu server	62
Appendix F: Installing Java	62
Appendix G: Installing Elasticsearch.....	63
Appendix H: Installing Logstash	64
Appendix I: Installing Kibana.....	66
Appendix J: Creating Index pattern in ELK	66
Appendix K: Installing Suricata.....	67
Appendix L: Installing Filebeat	68

List of Tables

2. 1 Types of firewalls	16
2. 2 Types of Monitoring System	20

List of Figures

3. 1	System	Design
.....	24	3. 2 Topology map
.....	26	
4. 1	Host-only	network
.....	30	4. 2 NAT network
.....	30	4. 3 Pfsense VM
settings	31	4. 4 Pfsense VM
startup interface	31	4. 5 Pfsense web
interface	32	4. 6 Ubuntu
server VM settings	32	4. 7 Ubuntu
server startup interface	33	4. 8 IP
address of the ubuntu server	33	4. 9
Web application configuration file	34	4.
10 web application files	34	4.
11 UFW status	34	4.
12 Web application	35	
4. 13 Elasticsearch directory	35	
4. 14 Elasticsearch configuration	35	
4. 15 jvm.options.....	36	
4. 16 Logstash directory	36	
4. 17 Logstash configuration file	36	
4. 18 Kibana directory	37	
4. 19 Kibana configuration file	37	
4. 20 ELK web interface	37	
4. 21 Suricata directory	38	
4. 22 Suricata configuration file	38	
4. 23 Suricata in var	38	
4. 24 Filebeat directory	38	
4. 25 Filebeat configuration file	39	
4. 26 Pfsense port forwarding	40	
4. 27 Pfsense rule	40	
4. 28 Router interface	41	
4. 29 Router port forwarding	41	
4. 30 DNS settings	42	
5. 1 Apache2 status	43	
5. 2 Elasticsearch status	43	
5. 3 Logstash status	44	
5. 4 Kibana status	44	
5. 5 Access logs	44	
5. 6 Error logs	45	
5. 7 Suricata status	45	
5. 8 Filebeat status	46	

5. 9 eve.json	46
5. 10 Filebeat log.....	47
5. 11 Website simulation test	47
5. 12 Simulation result	48
5. 13 Network monitoring	48
A. 1 Index pattern	69

List of Abbreviations

IDS – Intrusion Detection System

DoS – Denial of service

TTP - Tactics, Techniques, Procedures DEC

- Digital Equipment Corporation

IPS – Intrusion Prevention System

UTM – Unified Threat Management

IoT – Internet of Things

IP – Internet Protocol

NGFW – Next Generation Firewall

VPN – Virtual Private Network

IT – Information Technology

ML - Machine Learning

ELK – Elasticsearch, Logstash, Kibana

SIEM – Security Information and Event Management

LAN – Local Area Network

WAN – Wide Area Network

DNS – Domain Name System

CNAME – Canonical Name

NAT – Network Address Translation

VM – Virtual Machine

RAM – Random Access Memory

CPU – Central Processing Unit

UFW – Uncomplicated Firewall

APT – Advanced Package Tool

APTs - Advanced persistent threats

AI - Artificial Intelligence

SOAR – Security, Orchestration, Automation, Response

AWS – Amazon Web Services

A – Address

HTTP - Hypertext Transfer Protocol

HTTPS - Hypertext Transfer Protocol Secure

CHAPTER 1

1. Introduction

Network security is crucial for protecting information systems from unauthorized access, attacks and problems. This introduction highlights how important it is to put strong safety measures in place, like intrusion detection systems (IDS), firewalls, and monitoring tools, for the purpose to ensure the security of network environments.

1.1 Background of research

In today's digitally interconnected world, the protection of organisational assets and sensitive information has become a very important concern. Protecting the availability, integrity, and confidentiality of data depends on network security, which forms the basis of total cybersecurity [5]. A proactive and multi-layered strategy to security is required as the technology landscape changes and so do the sophistication and frequency of cyber threats.

Two of the defensive architecture's most important components are firewalls and monitoring systems. Firewall act as the digital guardian that creates a clear barrier between the internal network of an organisation and the hostile, public internet [7]. Firewalls prevent virus infiltration, denial-of-service (DoS) attacks, and unauthorised access by closely examining all incoming and outgoing network traffic against pre-established security rules [8]. Their implementation throughout the network levels, from the application layer to the basic network layer, offers a thorough defence-in-depth approach [4].

Although firewalls are highly effective in blocking illegal access, monitoring systems provide a proactive method of detecting and responding to threats [2]. Strictly monitoring network traffic, system logs, and security events, these systems look for errors, suspicious activity, and possible security breaches. Monitoring systems let organisations identify hazards in real-time,

minimising their impact and allowing quick response actions. Companies manage this through using advanced data analysis and recognition of patterns techniques [6].

For a strong security posture to be created, firewalls and monitoring systems must work in combination. The first line of defence is a firewall, which stops harmful traffic before it enters the network. But no security system is perfect, and intelligent opponents might figure out how to get around these barriers. By continuously examining network behaviour for indications of compromise, monitoring systems work in combination with firewalls to help organisations detect and deal with attacks that have gotten past their initial defences [3].

Moreover, incident response and investigation are made easier by the integration of firewalls and monitoring systems. Monitoring systems can give security teams important information on the tactics, techniques and procedures (TTPs) of an attacker during a security incident, allowing them to contain the breach and put countermeasures in place [1]. Also, strict post-incident investigations to identify underlying causes and prevent recurrence are likely to benefit from the detailed logs produced by both systems.

Network security needs to take a dynamic and flexible strategy considering the changing threat landscape. Strong firewall and monitoring systems are essential for organisations, and they must be frequently updated, configured correctly, and linked with other security measures [9]. Through the integration of these technologies with a solid security culture and training for employees, organisations can considerably enhance their ability to defend themselves against attacks and safeguard their valuable assets.

Cyber-attacks continue to provide a serious threat to companies globally, despite significant advancements in firewall and monitoring technologies. Investing in employee training, implementing a layered security approach, and staying informed of emerging risks and remedies are all necessary for firms to effectively combat these dangers [9]. For businesses to reduce risks and keep a solid security posture, incident response plans and routine security audits are also important.

1.2 Research Questions

The main research questions of this thesis are:

- In comparison to other firewall choices, which open-source firewall is the best?

- To be able to create an accurate and expandable monitoring system, how may firewall protection be combined with network and web application monitoring?
- What are the challenges and solutions in setting up the monitoring system for on-site real-time log monitoring and analysis?
- What is the best approach to keep a tab on the server's network?

1.3 Problem of statement

With the rise in cyberattacks and the need for strong network security, the goal of this project is to use open-source technologies to create a web server firewall and complete monitoring system. To provide a cohesive and scalable solution, the main difficulty addressed is the integration of many security tools, such as a firewall, an intrusion detection system (IDS), and a monitoring system. This solution needs to be affordable for small and medium-sized businesses, provide early threat detection, real-time information, and flexibility for future modifications.

1.4 Scope of study

Using open-source tools such as pfSense, the ELK stack, and Suricata IDS, this study constructs an integrated firewall and monitoring system for a web server. To improve network and online application security, it focuses on log management, real-time monitoring, and safe network configuration. Scalability of the system is evaluated to make sure it will support bigger networks or more complex security requirements. Comprehensive evaluation confirms that the system can identify and addressing risks, with the goal of offering small and medium-sized businesses a complete, adaptable, and affordable security solution.

1.5 Research Objectives

This study aims to develop, implement, and assess a comprehensive network security system that uses open-source technologies to incorporate intrusion detection, monitoring tools, and a firewall. By providing real-time threat detection, log management, and data visualization, the study aims to provide a solid and scalable solution that can secure a web server environment. The end goal is to develop a security infrastructure that is affordable and easily scalable for small and medium-sized businesses, improving their ability to defend against changing cyberthreats.

The main objectives are:

- To set up an open-source firewall to secure the network.
- To deploy a web application on a server.
- To implement monitoring system for comprehensive log management and analysis for a web application.
- To configure network monitoring for the server for any unusual activities.

1.6 Thesis Structure

This thesis is divided into six thorough chapters that systematically examine the development and application of an integrated network security system that combines intrusion detection, firewalls, and monitoring tools.

The introduction to the study is given in Chapter 1 along with background data, research questions, objectives, and a thesis structure breakdown. The literature review is covered in detail in Chapter 2, which includes the evolution of firewall technology, the historical development of network security, and the many types of firewalls. Also, it reviews important previous research and looks at intrusion detection system importance, monitoring tool types, and developments in monitoring systems. The research design, tools and network topology map are all included in the methods section of Chapter 3 of the study. It goes on explaining the methods used for data collecting and integration. The implementation of the system is the main topic of Chapter 4, which also covers the router rules, DNS, Suricata, ELK stack, Ubuntu server, Apache2, and environment setup. Chapter 5 includes network monitoring, web application monitoring, and simulation results. It focuses on testing and accuracy. Chapter 6 highlights the results, discusses the limitations, and makes recommendations for future research to expand the system's capabilities.

CHAPTER 2

2. Literature Review

2.1 Historical development of Network Security

Security was not a primary concern in the early days of the internet because digital ecosystems were still developing and mostly unexplored. Because of their smaller scale, threats to networks were more controlled and easier to understand. The foundation of robust digital security measures had not yet been fully established [37].

Digital defensive mechanisms started with the first attempts to secure digital networks, which include primitive firewalls created by pioneers like Digital Equipment Corporation (DEC). These primitive firewalls of the past distinguished between friends and enemies using simple packet filtering [54]. Although they worked well against the threats of the day, more intelligent opponents managed to quickly get around them.

But as the digital frontier expanded quickly, so did the number of increasingly complex threats. It was no longer possible to defend against these new threats with firewalls alone. Because of this, intrusion detection and prevention systems (IDS/IPS) were created [26]. These systems operate like watchful guardians, constantly monitoring the network for indications of possible intrusions.

A strong digital fortress was formed by integrating stateful firewalls [31] and IDS/IPS technologies to build a multi-layered protection system [46]. A comprehensive and strong defence against the constantly changing array of cyberthreats is guaranteed by this collaborative approach to cybersecurity.

The late 20th century saw the rise of the internet as an important commercial force that fundamentally changed how people lived, worked, and communicated [41]. While the digital revolution brought about unprecedented convenience, it also created a lucrative target for cybercriminals. Having strong defences became necessary. Unified Threat Management (UTM) has become an important comprehensive security solution, combining several protection systems into one unit [13].

The 21st century brought with it both new possibilities and difficulties. With the introduction of cloud computing [11], the digital world was completely transformed, and cross-border data transfers became possible. This dynamic movement was too much for traditional security

models to keep up with. Since then, cloud-based security systems have become more adaptable and flexible. With billions of devices connected, the Internet of Things (IoT) increased the attack surface and brought in a new era of connectivity [47].

Network security is a complex and dynamic field nowadays that requires a combination of human insight, technological expertise, and a thorough understanding of dangers. Human intelligence and resilience have played a crucial role in creating strong defences, from the earliest days of simple firewalls to the most advanced security systems of today [16]. The challenges will change along with the digital environment. Every new threat, though, offers a chance for defence strengthening, innovation, and the creation of a more secure digital future.

2.2 Evolution of Firewall Technologies

When the internet first started, security was simply an inquest. Due to their small size and privacy, networks faced relatively easy dangers. But the development of the firewall represented a major step forward in safeguarding these emerging digital domains [24].

In the past, firewalls—also known as packet-filtering firewalls—evaluated both incoming and outgoing network traffic to decide which packets, depending on protocols, IP addresses, and port numbers, to allow or reject [54]. These firewalls were limited in their ability to examine the contents of the data they were protecting, even though they were successful against common threats.

The difficulties faced by the internet expanded and developed along with it. Hackers discovered more advanced techniques to get past these first security measures. With the ability to track packets at an individual level and maintain the context of a virtual discussion, stateful inspection firewalls become an increasingly complex kind of protection [54].

As the digital landscape develops more, it offers both new opportunities and difficulties. While innovations like social media, cloud computing, and online shopping have altered our way of life and work, they additionally created new openings for hackers to take advantage. As a result, firewalls have evolved into application-layer firewalls, which are able to examine packet contents that are not limited to their surface layer [55]. With the help of deep packet inspection, these sophisticated firewalls can detect and thwart an even wider spectrum of threats, acting as gatekeepers with X-ray vision [17].

The introduction of Next-Generation Firewalls (NGFWs) signified a major advancement in firewall technology. These were not ordinary gatekeepers; instead, they were smart security

officers with advanced capabilities like user identity management, intrusion prevention, and application control [27]. It was like taking a single gatekeeper and turning them into a whole security team, with each person having their expertise. By identifying users, analysing traffic patterns, and detecting anomalies, NGFWs provide a strong defence against the constantly evolving threat landscape.

New technologies are always emerging in today's fast-paced digital environment, posing new difficulties for network security. Firewalls have once again had to adjust with the rise of cloud computing and the growing usage of mobile devices. To provide strong protection in the dynamic, international cloud environment, virtual firewalls [50] and cloud-native security solutions [49] have surfaced.

Human creativity in the face of constant challenges is demonstrated by the evolution of firewalls [16]. Firewalls is important for protecting our increasingly linked world, having developed from their simple origins as packet filters to sophisticated digital guardians. By continually enhancing their capabilities, firewalls can effectively protect our digital lives by growing more proactive, intelligent, and flexible as the threat landscape changes.

2.2.1 Different types of Firewalls

When it comes to strong security features without the added expense of licensing, open-source firewalls offer an excellent alternative for private solutions [42]. Community ownership drives these firewalls, enabling ongoing customization and enhancement. Since they are clear, affordable, and adaptable, they have become becoming increasingly common in both home and business environments. There are several open-source firewalls: pfSense, OPNsense, IPFire, ClearOS, Untangle, IPCop and Smoothwall Express.

Pfsense

A highly regarded open-source firewall and router program based on FreeBSD is called pfSense [39]. Famous for its flexibility, reliability, and vast feature set, pfSense is developed and maintained by Netgate. It provides load balancing, VPN support, advanced firewall features, and an interface that can be customised through a web-based graphical user interface. Because of its flexible structure, pfSense can be made to do more by utilizing an extensive number of plugins and packages, meeting a variety of security needs. It is an adaptable choice for network security because it covers a range of deployment circumstances, from small homes to big businesses.

OPNsense

With the focus on enhanced safety, performance, and user experience, OPNsense—a copy of pfSense—emerged in 2015 [45]. While it offers enhancements and new functionality, it is still built on FreeBSD and has many of the same essential functions as pfSense. OPNsense prioritizes a more modern and simpler user interface that is suitable for users with and without experience. It has features which includes an extensive reporting and analytics module, intrusion detection and prevention systems (IDS/IPS), and two-factor authentication. OPNsense's dedication to timely security patch updates and updates guarantees that it continues to be an effective firewall choice.

IPFire

Designing with simplicity, security, and flexibility in consideration IPFire is an open-source firewall system built on the Linux operating system [28]. Because of its modular nature, customers can customize the system to meet their own needs. The functionalities of IPFire include web proxy, VPN, stateful packet inspection, and IDS/IPS. Even for individuals with little knowledge of technology, setting up and handling it is simple thanks to its user-friendly online interface. By emphasizing quality of service (QoS), IPFire enables users to prioritize network traffic and ensure optimal performance for critical applications. This is one of its most notable characteristics. Its adaptability and reliability have been further strengthened by IPFire's frequent updates and active community.

ClearOS

The open-source operating system ClearOS was developed by ClearCenter with small enterprises and home networks in consideration [33]. By combining server, firewall, and gateway functions into one platform, it offers a complete network administration solution. Because of its user-friendly web interface and CentOS foundation, ClearOS is appropriate for users with different levels of technical proficiency. It has features including VPN support, antivirus software, intrusion detection, and content filtering. To be able to customize ClearOS to meet their specific needs, users can add more features through an app and extension marketplace. ClearOS is a popular option for small businesses looking for a unified network security solution because of its emphasis on simplicity and extensive feature set.

Untangle

An open-source firewall and unified threat management (UTM) platform for small and medium-sized enterprises is called Untangle [48]. Firewall, VPN, browser filtering, antivirus,

and application control are just a few of the security features it provides. Users may improve system performance and use of resources by installing only the functions they require thanks to Untangle's modular design. Users with different technical backgrounds can use it because of its easy-to-use web-based interface, which makes configuration and management simpler. Administrators can gain insights into network activities and any safety issues with the help of Untangle's comprehensive reporting and analytics capabilities. Untangle is a flexible and efficient firewall solution because of its concentration on user-friendliness, extensive security features, and modular architecture.

IPCop

Another Linux-based open-source firewall program is called IPCop, and it's intended to give users an easy-to-use network connection security solution [36]. Features which IPCop provides include web proxy support, VPN support, and stateful packet inspection. Setting it up and managing it is simple, even for individuals with no technical expertise because to its simple web-based interface. If reliability and simplicity of usage are critical, IPCop is particularly appropriate for small to medium-sized networks. IPCop is still a good choice for anybody looking for a simple to operate reliable firewall, even though development has slowed down significantly.

Smoothwall Express

An initiative aimed at enhancing the usability and reliability of firewalls is called Smoothwall Express [18]. Autonomous packet inspection, web proxies, and VPN support are just some of its features. The web-based interface of Smoothwall Express simplifies configuration and management easier, and it is made with ease of use in mind. For small businesses and household networks that need a simple yet powerful firewall solution, it works particularly well. Users looking for a no-frills firewall solution might find Smoothwall Express attractive due to its focus on simplicity and reliability.

Table 2. 1 Types of firewalls

Firewall Type	Key Features	Type of Filtering
pfSense	Stateful packet inspection, VPN support, traffic shaping	Stateful, Packet Filtering
OPNsense	Intrusion detection, two-factor authentication, traffic monitoring	Stateful, Packet Filtering, IDS/IPS
IPFire	Web proxy, IDS, VPN support	Stateful, Deep Packet Inspection

ClearOS	Gateway antivirus, content filtering, intrusion detection	Stateful, Proxy Filtering, Application Layer
Untangle	Application control, web filtering, intrusion prevention	Stateful, Application Layer Filtering
IPCop	Intrusion detection, VPN, real-time monitoring	Stateful, Packet Filtering
Smoothwall Express	Web filtering, VPN, traffic shaping	Stateful, Proxy Filtering

Network security can be achieved at a reasonable price with reliable, adaptable, and flexible open-source firewalls such as pfSense, OPNsense, IPFire, ClearOS, Untangle, IPCop and Smoothwall Express. Due to their unique characteristics and features, these firewalls can be used for a range of safety needs and deployment situations. There is an open-source firewall solution that can meet the unique needs and offer thorough security against cyber threats, whether for home networks, small businesses, or large enterprises.

2.3 Monitoring System

Due to the increasing complexity of IT infrastructures and the demand for real-time insights into system security and performance, monitoring systems have seen significant changes over the past few decades. Throughout this progression, basic, reactive monitoring systems have provided way to developed, proactive ones with programmable responses and predictive analytics [30].

2.3.1 Advances in Monitoring Systems

There are many advancements in monitoring systems:

Real-Time Monitoring and Alerting: Conventional surveillance systems collected data by batch processing and regular polling, allowing insights and answers to be acquired slowly. For instant visibility into system performance and security, modern monitoring systems, on the other hand, take advantage of immediate data gathering and streaming technology [32]. Recognizing and addressing problems before they affect users requires these real-time capabilities.

Machine Learning and Predictive Analytics: Monitoring system's capabilities were transformed with the introduction of machine learning (ML). To provide predictive analytics, machine learning algorithms analyse both historical and current data to find trends and

anomalies [43]. By doing this, businesses can reduce downtime and increase system reliability by anticipating possible problems and taking preventative steps to fix them.

Scalability and Flexibility: Monitoring systems may now easily grow with expanding IT infrastructures thanks to advances in cloud computing [25]. Because cloud-based monitoring solutions are elastic, they lack substantial upfront spending in software and hardware, allowing businesses to monitor massive and dynamic settings.

Integration and Automation: Modern monitoring solutions are made to easily work with other IT management platforms and applications. With the help of this integration, automated workflows—in which the automatic execution of tasks like resource scaling, service restarts, and administrator alerts—are made possible by data monitoring [14]. Automation reduces reaction times, reduces manual intervention, and improves system resilience overall.

Comprehensive Visibility and Unified Monitoring: The key element of advanced monitoring systems is comprehensive visibility across the entire IT stack [23]. Databases, servers, networks, applications, and other sources of data are all combined into one pane of glass through unified monitoring platforms. By providing contextual insights into the health and performance of the system, this holistic approach accelerates troubleshooting and improves decision-making.

2.3.2 Types of monitoring Systems

There are many types of monitoring systems:

ELK Stack (Elasticsearch, Logstash, Kibana)

A well-liked open-source monitoring tool known for flexibility and potent analytics features is the ELK Stack [22]. It is made up of three primary parts:

Elasticsearch: An analytics and distributed search engine that indexes and saves log data for quick analysis and retrieval.

Logstash: A pipeline for data processing that receives, processes, and forwards data to Elasticsearch. Many input sources are supported by this, which also has real-time data filtering and enhancement capabilities.

Kibana: Elasticsearch log data analysis is made easier, an interactive dashboard and visualization tool.

Because of its capacity to handle massive amounts of log data, deliver real-time insights, and provide customisable dashboards for a range of monitoring requirements, the ELK Stack is frequently used for server monitoring.

Prometheus

Scalability and dependability are key features of Prometheus, an open-source monitoring and alerting toolbox [51]. Metrics are collected from configured targets at predetermined intervals and stored in a time-series database. Large-scale and dynamic environment monitoring is suited for Prometheus due to its strong querying and alerting abilities.

Nagios

Nagios is a popular open-source monitoring system that offers thorough server, network device, and application monitoring for IT infrastructure components [40]. Its modular architecture facilitates simple modification and growth through plugins, and it includes robust alerting and reporting features.

Zabbix

Zabbix is an open-source, enterprise-grade monitoring solution with graphing, alerting, and real-time monitoring tools [38]. It provides a highly configurable web-based interface for managing and setting up monitoring tasks, and it supports a wide range of monitoring methodologies, including agent-based and agentless monitoring.

Grafana

Grafana connections with Prometheus, Elasticsearch, and InfluxDB, among other data sources. It is an open-source platform for observability and monitoring. For analysing and interpreting monitoring data, it provides robust dashboards and visualization abilities [15]. A popular option for unified monitoring and analytics is Grafana because of its extensibility and support for many data sources.

Splunk

Large-scale machine-generated data processing and analysis are characteristics of Splunk, a proprietary monitoring and analytics platform [19]. Because of its sophisticated search, reporting, and visualization features, it is a good choice for security analytics and comprehensive IT monitoring.

Sensu

An open-source monitoring tool called Sensu is made for environments that are cloud-native and dynamic. Its flexibility allows for wide customization and interaction with other IT management tools [34]. It collects and processes monitoring data using a publish-subscribe approach.

Table 2. 2 Types of Monitoring System

Monitoring System	Key Features	Type of Monitoring
ELK Stack	Centralized logging, real-time analysis, scalable	Log and Event Monitoring
Prometheus	Time-series data storage, multidimensional model	Time-series Monitoring
Nagios	Host and service monitoring, notification system	Host and Service Monitoring
Zabbix	Real-time monitoring, distributed architecture	Network and Application Monitoring
Grafana	Custom dashboards, real-time visualization	Metrics Visualization, Timeseries Monitoring
Splunk	Real-time data indexing, predictive analytics	Log and Event Monitoring
Sensu	Event-based alerting, scalable architecture	Infrastructure Monitoring

Monitoring systems can now provide immediate feedback, predictive analytics, scalability, integration, and complete visibility thanks to major advancements in the field. A variety of monitoring systems, like Sensu, Zabbix, Prometheus, Nagios, Splunk, Grafana, and ELK Stack, provide unique features and advantages suited to various monitoring needs. Specific criteria, such as scalability, simplicity, integration capabilities, and the makeup of the IT environment, influence the choice of monitoring system.

2.4 Intrusion Detection Systems

Modern cybersecurity frameworks must involve intrusion detection systems (IDS), which are essential for monitoring, identifying, and reacting to unauthorized or threatening activity in an IT environment [26]. IDS is crucial for protecting servers—which frequently include sensitive data and important applications—against many types of cyberattacks. This section examines the types, functions, and significance of intrusion detection systems (IDS) for servers,

emphasising significant developments and their applicability in the current security environment.

2.4.1 Importance of Intrusion Detection Systems

Servers that form the foundation of many organizational processes, are often the target of cyberattacks [10]. The growing complexity of attacks demands strong security protocols that go beyond firewalls and antivirus programs. By constantly monitoring server activity and instantly detecting possible security breaches, intrusion detection systems offer an extra degree of protection. They play a crucial role in the identification of numerous kinds of attacks, including denial-of-service attacks, malware infections, illegal access, and data exfiltration.

The main benefit of intrusion detection systems (IDS) is the ability to deliver early alerts about security events, allowing administrators to promptly implement corrective actions [44]. This proactive approach lowers the total risk to the IT infrastructure of the company and lessens the harm that security breaches might bring. IDS can provide insightful information about attack patterns and weaknesses, which helps security postures continue to evolve.

2.4.2 Advances in Intrusion Detection Systems

There are many advancements in Intrusion Detection Systems:

Artificial Intelligence and Machine Learning: IDS systems are increasingly incorporating AI and ML technologies to improve detection precision and minimise false positives [20]. These technologies improve the effectiveness of intrusion detection systems (IDS) by allowing them to learn from past data and respond to new attack patterns.

Integration with Security Information and Event Management (SIEM): SIEM systems and modern IDS are frequently connected to provide a single, organizational picture of security events [35]. SIEM systems provide complete threat intelligence by collecting and matching data from multiple sources, such as intrusion detection systems (IDS), to identify complex patterns of attacks.

Behavioural Analysis: The goal of behavioural analysis addresses is to understand how users and applications act in the server environment [57]. IDS can detect variations from typical behaviour patterns that can point to malicious activity. Advanced persistent threats (APTs) and insider risks can both be found with behavioural analysis.

Cloud-based IDS: Cloud-based intrusion detection systems (IDS) are beginning to offer security monitoring for cloud environments in combination with the growing popularity of cloud computing [53]. These technologies make effective use of the cloud's scalability and flexibility to monitor distributed and dynamic server designs.

2.5 Related Previous Research Paper

Some of the previous related papers like this topic are:

1. Surveying Trust-Based Collaborative Intrusion Detection: State-of-the-Art, Challenges and Future Directions

In the research paper the authors Wenjuan Li, Weizhi Meng and Lam For Kwok gives a thorough analysis of several Intrusion Detection Systems (IDS) and their approaches [52]. It covers the history of IDS models, their shortcomings, and the development of more sophisticated systems. It provides a thorough overview of intrusion detection systems (IDS).

In this thesis, the model goes one step further by integrating IDS with firewalls and real-time monitoring through the ELK stack. This approach provides a more comprehensive approach to network security and addresses the need for a single system that not only recognizes intrusions but also stops them through firewall integration and offers comprehensive analysis through continuous monitoring.

2. Network Security and Next-Generation Firewalls

The authors D. Freet and R. Agrawal examines the development of firewalls, with a focus on the potential and problems brought up by Next-Generation Firewalls (NGFWs) [21]. The importance of integrated threat intelligence and application-level security is emphasized.

The concept discussed in this paper integrates conventional firewalls with IDS and monitoring technologies. By adding IDS and monitoring tools that can detect attacks that might get past NGFWs, it overcomes the flaws of NGFWs and places a weight on combining different security technologies to provide an improved solution.

3. A Comparative Analysis of Open-Source Network Monitoring Tools

The author Ali Al Shidhani, Khalil Al Maawali, Dawood Al Abri and Hadj Bourdouden research analyses a few free and open-source network monitoring tools, highlighting their

functionalities, features, and overall performance [12]. It offers insights into the advantages and disadvantages of every instrument.

While this research paper focuses on monitoring tools, the thesis model's use of the ELK stack provides a more reliable method of tracking and evaluating network traffic, particularly when paired with an intrusion detection system (IDS) like Suricata. This model offers an improved monitoring solution by combining security monitoring through IDS with network monitoring, giving better visibility and control.

4. Research and implementation on snort-based hybrid intrusion detection system

In this research the authors Yu-Xin Ding, Min Xiao and Ai-Wu Liu provide a hybrid intrusion detection system (IDS) that minimises false positives and improves detection accuracy by combining machine learning approaches with the open-source IDS Snort [56].

In contrast to this paper, which focuses on improving IDS through machine learning, this model of the thesis places more emphasis on how IDS may be combined with firewalls and monitoring technologies to create a multi-layered security approach. This concept offers a more complete security solution that addresses detection, prevention, and response by extending the capabilities of a conventional IDS and combining it with other security tools.

5. Challenges and Directions in Security Information and Event Management (SIEM)

The authors Marcello Cinque, Domenico Cotroneo and Antonio Pecchia analyses many SIEM frameworks and systems, highlighting their features, architecture, and limitations in this paper [29]. It goes into how SIEMs combine information from several sources to offer thorough security monitoring.

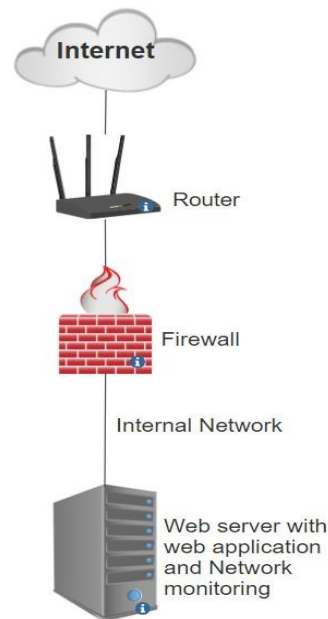
But this dissertation model discusses the need for a more reasonable, yet useful, use for traditional SIEM systems, making comprehensive security monitoring more readily available and simpler to implement. It can be thought of as a lightweight SIEM, applying the ELK stack for log management and analysis while integrating IDS and firewall data for a more thorough security approach.

CHAPTER 3

3. Methodology

3.1 Research Design

The network security and monitoring system is designed to provide strong protection and thorough monitoring of network traffic for a web application. Figure 3. 1 System Design



Webserver: The web server hosts the web application and all necessary services for monitoring that application.

Internal network: The network lane is used to host the web application locally within the inner network.

Firewall: This next-generation firewall is designed to protect web servers from unauthorized users and malicious requests.

Router: It plays a crucial role in directing traffic on the internet, ensuring that data sent from one device reaches its correct destination.

Internet: The internet or any external sources that seek to access the web server

3.2 Tools used

The tools used to set up this system are:

Pfsense: pfsense is an open-source firewall and router software that offers many features ideal for network security. The pfSense serves as a powerful firewall, serving as the primary defence against external threats [39]. The pfsense version used is pfsense 2.7.0. It is typically installed on a dedicated hardware device or virtual machine to effectively manage and filter all incoming and outgoing network traffic.

Ubuntu Server: Ubuntu Server is a Linux-based operating system recognized for its exceptional stability, security, and flexibility [59]. The ubuntu server version used is ubuntu 24.04 LTS. In this, Ubuntu Server serve as the core operating system for setting web application and additional tools, including the ELK Stack and Suricata, is installed and carefully configured.

Apache2: The Apache2 web server software is one of the most popular and widely utilized web hosts in the world [58]. By handling requests and sending responses, which include web pages, graphics, and other types of content, across the internet, this potent technology is essential to effectively serving web content to users.

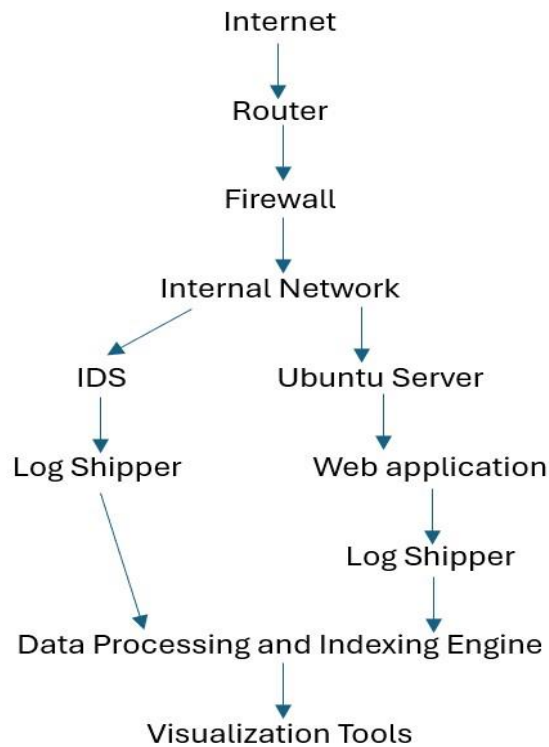
ELK stack: It is comprised of Elasticsearch, Logstash, and Kibana, is a robust open-source toolset designed for the management and analysis of log data [22]. In this the ELK Stack is implemented to oversee network traffic, analyse logs from multiple sources, and present data visually to detect potential security risks.

Filebeat: Filebeat is a lightweight log shipper that is utilized to efficiently forward and centralize log data [61]. In this configuration, it is employed to gather logs from Suricata, and then transmit them to Elasticsearch for further processing.

Suricata: Suricata is an open-source network intrusion detection system (NIDS), intrusion prevention system (IPS), and network security monitoring (NSM) engine [60]. It is utilized in this configuration to monitor network traffic for any indications of intrusion or malicious activity.

3.3 Network Topology Map

Figure 3. 2 Topology map



Firewall: The firewall serves as the main gatekeeper, allowing only authorized traffic to enter the internal network by filtering traffic according to security regulations. For additional insight, it tracks every traffic activity as well.

Ubuntu Server: Hosting the web server and setting monitoring tools, the Ubuntu Server acts as the hub of the internal network. It executes the web application that users interact with and is protected from unwanted access.

Intrusion Detection System (IDS): The IDS keeps an eye on all network traffic that enters the internal network and searches for any unusuality's that could demonstrate malicious behaviour, including brute force attacks or port scanning.

Log Shipper: This component gathers logs from the web server and intrusion detection system and sends them to the data processing engine. This guarantees the centralization of all pertinent security data for analysis.

Data Processing and Indexing Engine: After obtaining logs from the log shipper, the data processing and indexing engine processes the logs to derive valuable information and then indexes them for simple searching and analysis. This element is essential for responding to security problems and seeing patterns over time.

Visualization Tool: To provide real-time dashboards and visualizations of network activity and security alerts, the visualization tool communicates with the data processing engine.

Administrators can quickly recognize and address such threats because of it.

3.4 Integration process

There are a few essential actions that must be taken to integrate these parts into an effective system:

Set up firewall: Pfsense firewall is installed on a Virtual machine. LAN and WAN network is assigned after installation. Security rules are implemented to govern the movement of data, determining which types of traffic are permitted or blocked based on criteria such as IP addresses, protocols, and port numbers.

Set up webserver: Ubuntu server is installed on a Virtual machine and configured in the internal network of the firewall. It hosts the web application, managing the backend and frontend phases and is configured to manage error and access logs. And all the monitoring tools and services such as ELK stack, Suricata, Filebeat are installed on the server.

IDS: Suricata is set up in the web server to monitor the server's internal network. It is set up to examine every bit of network traffic that flows through it, looking for any trends that might point to malicious behaviour.

Log Shipper Deployment: Log shipper such as Filebeat and Logstash is used to forward logs. Network logs are collected by Filebeat and sent straight to Elasticsearch. Advanced data manipulation is made possible by Logstash, which ensures that web application logs are correctly structured and enhanced with extra information before being stored.

Data Processing and Visualization Setup: A potent tool for processing, storing, and visualizing data obtained from web servers and IDS is the ELK Stack, which consists of Elasticsearch, Logstash, and Kibana. Administrators can maintain surveillance on their systems in real time and respond to security events with effectiveness due to this strong structure.

Set up Router configuration: Data flow between the internal network, the internet, and other networks is controlled by the router. Port forwarding rules are set up on the router. By ensuring that traffic passes through the firewall, these rules enable the firewall to implement security regulations.

DNS set up: The Domain Name System (DNS) is a necessary instrument for connecting a web application's domain name to its public IP address, allowing online access to the program. This can only be accomplished by first registering a domain name with a domain registrar. DNS

records are next set up to point the domain name towards the public IP address. These records include CNAME (Canonical Name) and A (Address). It is important to remember that the propagation of DNS record updates throughout the internet could take some time after they are configured.

3.5 Data processing method

Data processing, which oversees gathering, analysing, storing, and displaying logs produced by different components including Suricata, Filebeat, Logstash, Kibana, and Elasticsearch, is an essential component of the network security model. The process may be broken down into two primary data flows: one uses Suricata for network monitoring and another uses Logstash for web application monitoring. The flow of data through this model is explained in detail in the following.

NETWORK MONITORING

As an Intrusion Detection System (IDS), Suricata is a powerful open-source network threat detection engine. It maintains a close eye on network traffic in real time, detecting suspicious activity and keeping detailed records of all it finds. Suricata's logs provide useful details regarding network incidents, possible security risks, and unusuality's observed in the traffic. These logs are essential for determining potential breaches and evaluating the state of network security. Filebeat gathers and sends log data in the capacity of a log shipper. Filebeat is set up especially to watch the Suricata log files in this case. It reads the logs that Suricata produces and sends them to the pipeline's next step.

Elasticsearch works with a data visualization tool called Kibana. Filebeat sends the logs straight to Kibana during this process so they may be seen. But Elasticsearch handles data storing and searching, and Kibana mostly serves as a frontend interface, while Kibana is used to display the logs, Elasticsearch is where they are kept.

The log data is stored and indexed by Elasticsearch, a powerful search and analytics engine. Elasticsearch indexes and stores the data after it is seen in Kibana, making searching, analysis, and querying more effective. This setup makes it easier to analyse network traffic in the past and in present time.

WEB APPLICATION MONITORING

Strong data processing pipeline Logstash is great at consuming data from sources and then delivering it to Elasticsearch. In this configuration, Logstash is essential for handling the web

application's access and error logs. Logstash is specifically set up to collect the web application's error and access logs. These logs provide insightful information on the actions of the web servers, including specifics about requests sent to the server, faults encountered, and other interesting events.

Once the logs are processed, Logstash forwards them to Kibana for visualization. Kibana is used visualizing and interacting with the data, offering dashboards, charts, and other visualization tools to help make sense of the logs.

After processing, the logs are kept in Elasticsearch, which indexes and allows users to search the data. Elasticsearch facilitates in-depth monitoring and reporting on web server activity by allowing users to run complex searches on the logs that have been stored. Kibana makes use of Elasticsearch's capabilities to offer analysis of past data and insights in real time.

CHAPTER 4

4. Implementation

4.1 Setting up Environment

Oracle VM VirtualBox is the virtual environment in which the system runs [65]. Virtual networks, such as a NAT network and a host-only network with the IP address 192.168.1.100, are set up before virtual machines (VMs) are created.

Figure 4. 1 Host-only network

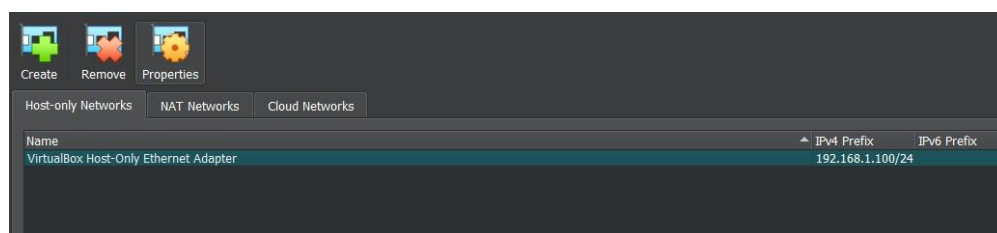
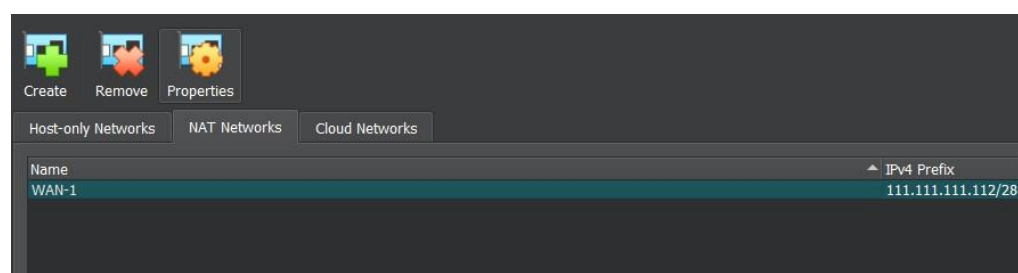


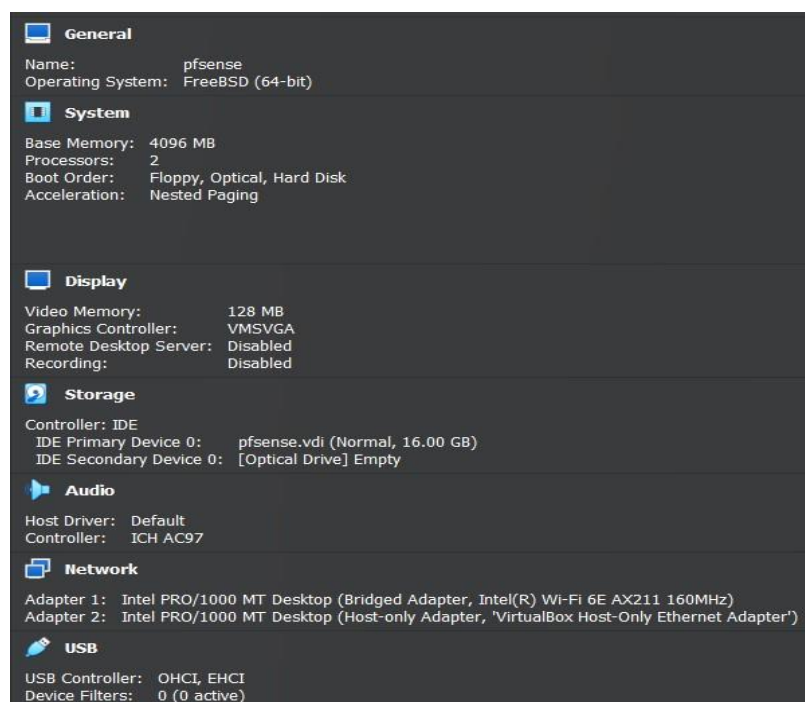
Figure 4. 2 NAT network



4.2 Setting PfSense

The pfsense ISO image is downloaded from pfsense official website [66]. The next step is to build up a virtual machine for pfSense with the proper RAM, CPU, and storage allocation parameters. It's critical to make two network adapters: one for internal network connections and another for internet connections. All the settings used are shown in the figure below.

Figure 4. 3 Pfsense VM settings



The virtual machine (VM) is given the right LAN and WAN interfaces after the pfSense firewall has been launched.

Figure 4. 4 Pfsense VM startup interface

```

--- 192.168.1.102 ping statistics ---
3 packets transmitted, 3 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 2.893/4.204/5.675/1.141 ms

Press ENTER to continue.

VirtualBox Virtual Machine - Netgate Device ID: 982eeee375b1c8b946ed

*** Welcome to pfSense 2.7.0-RELEASE (amd64) on pfSense ***

WAN (wan)      -> em0      -> v4/DHCP4: 192.168.0.21/24
LAN (lan)      -> em1      -> v4: 192.168.1.1/24

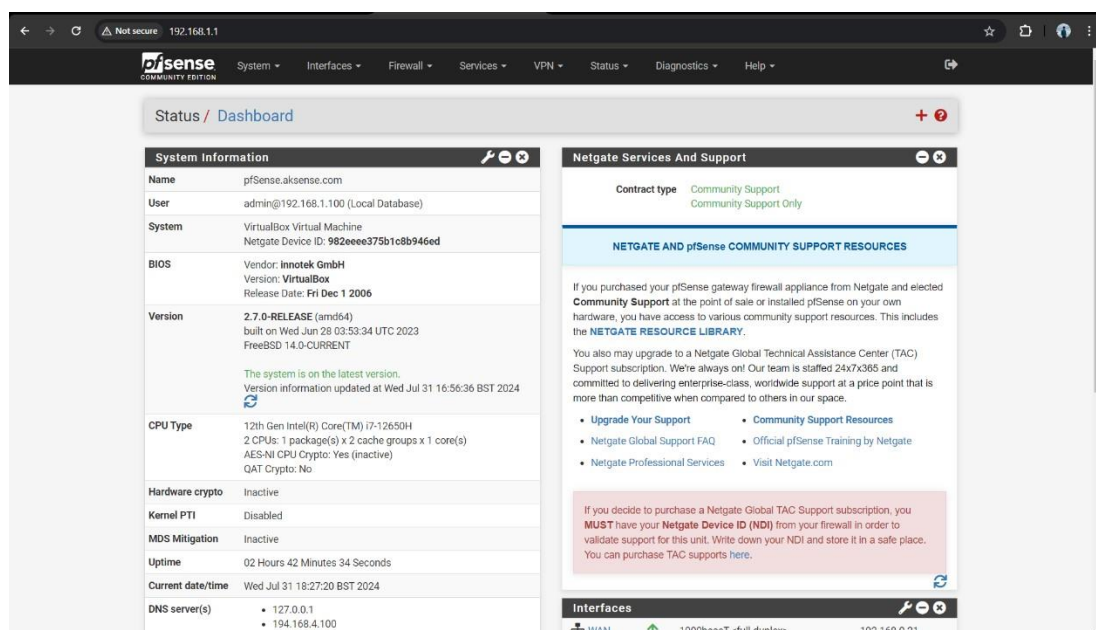
0) Logout (SSH only)          9) pfTop
1) Assign Interfaces          10) Filter Logs
2) Set interface(s) IP address 11) Restart webConfigurator
3) Reset webConfigurator password 12) PHP shell + pfSense tools
4) Reset to factory defaults  13) Update from console
5) Reboot system              14) Enable Secure Shell (sshd)
6) Halt system                 15) Restore recent configuration
7) Ping host                   16) Restart PHP-FPM
8) Shell

Enter an option:

```

The virtual machine's LAN address can be used to access the pfSense web interface from the host system using the default username “admin” and password “pfsense”. Then, the password is changed “admin”.

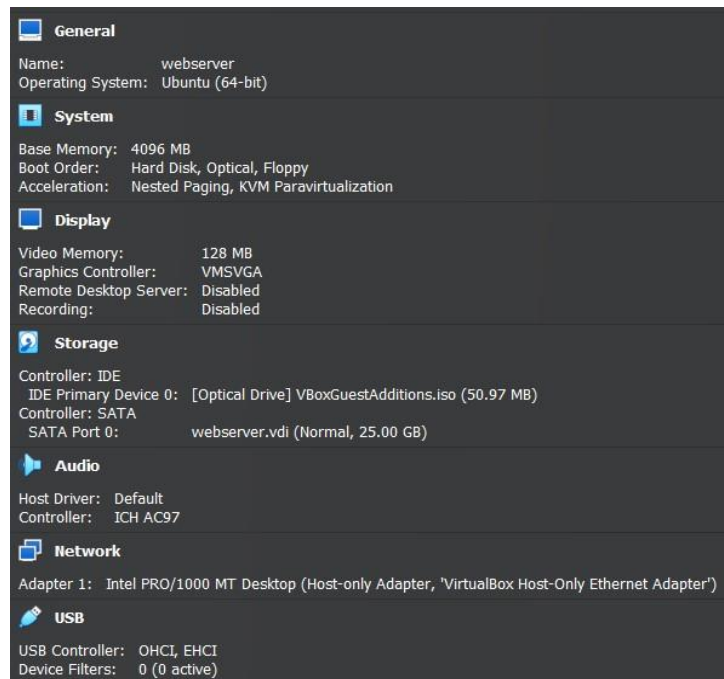
Figure 4. 5 Pfsense web interface



4.3 Setting Ubuntu server

The ubuntu server ISO image is downloaded from the ubuntu official website [64]. After downloading, the proper allocation settings should be applied for creating a virtual machine (VM). Connecting to the internal network also requires the creation of a network adapter. The settings are shown in the figure below.

Figure 4. 6 Ubuntu server VM settings



The Ubuntu server installation instructions are followed, after the virtual machine (VM) has started. "Webserver" is the username and "webserver" is the password when creating a user account. The virtual machine reboots when the installation is finished. Due to its connection to the internal network, the Ubuntu server's IP address is 192.168.1.102 (Refer Appendix A).

Figure 4. 7 Ubuntu server startup interface

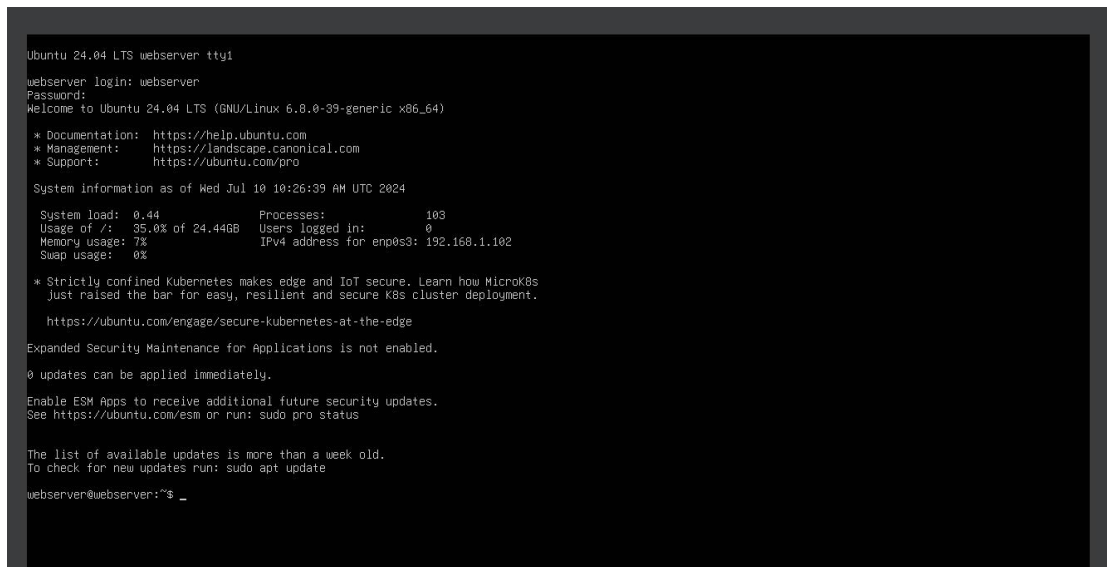


Figure 4. 8 IP address of the ubuntu server


```

webserver@webserver:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 08:00:27:da:10:eb brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.102/24 metric 100 brd 192.168.1.255 scope global dynamic enp0s3
        valid_lft 7046sec preferred_lft 7046sec
    inet6 fe80::a00:27ff:feda:10eb/64 scope link
        valid_lft forever preferred_lft forever
webserver@webserver:~$ _

```

4.4 Setting Apache2

The server that hosts the web application has the Apache2 server installed on it (Refer Appendix B). While the web application itself is stored in the var directory, the server's etc directory contains the configurations needed to host using Apache2. To meet the requirements of the web application, all default Apache2 websites and configurations will be changed. The default files will be replaced with those needed for the web application files called "webapp" (Refer Appendix C) and its configuration file named "test.project.final.conf" (Refer Appendix D).

Figure 4. 9 Web application configuration file

```

root@webserver:/etc# cd apache2
root@webserver:/etc/apache2# ls
apache2.conf  conf-available  conf-enabled  envvars  magic  mmds-available  mmds-enabled  ports.conf  sites-available  sites-enabled
root@webserver:/etc/apache2# cd sites-enabled
root@webserver:/etc/apache2/sites-enabled# ls
test.project.final.conf
root@webserver:/etc/apache2/sites-enabled# cat test.project.final.conf
<VirtualHost *:80>
    # The ServerName directive sets the request scheme, hostname and port that
    # the server uses to identify itself. This is used when creating
    # redirection URLs. In the context of virtual hosts, the ServerName
    # specifies what hostname must appear in the request's Host: header to
    # match this virtual host. For the default virtual host (this file) this
    # value is not decisive as it is used as a last resort host regardless.
    # However, you must set it for any further virtual host explicitly.
    #ServerName www.example.com

    ServerAdmin webmaster@localhost
    ServerName theanugrah.com
    ServerAlias www.theanugrah.com
    DocumentRoot /var/www/webapp/
    ErrorLog /var/www/webapp/logs/error.log
    CustomLog /var/www/webapp/logs/access.log combined
    <Directory /var/www/webapp/>
        Options -Indexes +FollowSymLinks
        AllowOverride All
        Require all granted
    </Directory>
    # Available loglevels: trace8, ..., trace1, debug, info, notice, warn,
    # error, crit, alert, emerg.
    # It is also possible to configure the loglevel for particular
    # modules, e.g.
    #LogLevel info ssl:warn

    #
    ErrorLog ${APACHE_LOG_DIR}/error.log
    CustomLog ${APACHE_LOG_DIR}/access.log combined

    # For most configuration files from conf-available/, which are
    # enabled or disabled at a global level, it is possible to
    # include a line for only one particular virtual host. For example the
    # following line enables the CGI configuration for this host only
    # after it has been globally disabled with "a2disconf".
    #Include conf-available/serve-cgi-bin.conf
</VirtualHost>
root@webserver:/etc/apache2/sites-enabled# _

```

Figure 4. 10 web application files

```
root@webserver:~# cd /var/www/webapp
root@webserver:/var/www/webapp# ls
about.html  assets  contact.html  css  index.html  imgs
root@webserver:/var/www/webapp#
```

The UFW (Uncomplicated Firewall) is enabled and has full permission to filter incoming requests directly on the server (Refer Appendix E).

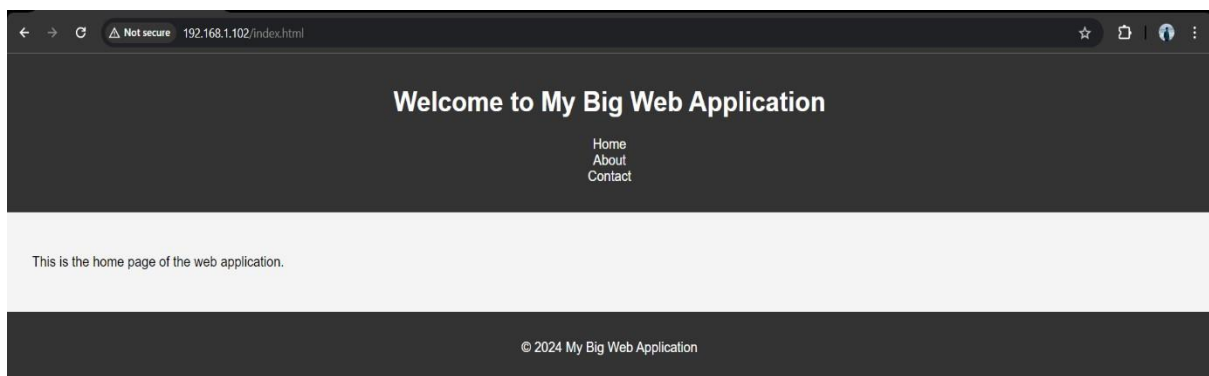
Figure 4. 11 UFW status

```
root@webserver:/etc# cd
root@webserver:~# ufw status
Status: active

To Action From
--
22/tcp ALLOW Anywhere
443 ALLOW Anywhere
8080 ALLOW Anywhere
Apache Full ALLOW Anywhere
OpenSSH ALLOW Anywhere
5601/tcp ALLOW Anywhere
9200/tcp ALLOW Anywhere
5601 ALLOW Anywhere
22/tcp (v6) ALLOW Anywhere (v6)
443 (v6) ALLOW Anywhere (v6)
8080 (v6) ALLOW Anywhere (v6)
Apache Full (v6) ALLOW Anywhere (v6)
OpenSSH (v6) ALLOW Anywhere (v6)
5601/tcp (v6) ALLOW Anywhere (v6)
9200/tcp (v6) ALLOW Anywhere (v6)
5601 (v6) ALLOW Anywhere (v6)
```

After the web application's configuration file is enabled, the IP address of the server can be used to access it across the internal network.

Figure 4. 12 Web application



4.5 Setting ELK stack

The server's installation of the ELK stack went quite successfully. First, Java needed to be set up and the public signing key needed to be installed (Refer Appendix F). The Elasticsearch installation was then completed with the creation of the Elasticsearch APT repository. To properly run the server, Elasticsearch configuration is carefully configured (Refer Appendix G).

Figure 4. 13 Elasticsearch directory

```
root@webserver:/etc# javac --version
javac 21.0.3
root@webserver:/etc# cd elasticsearch
root@webserver:/etc/elasticsearch# ls
elasticsearch.kubernetes  elasticsearch.yml  jvm.options.d  role_mapping.yml  users
elasticsearch-plugins.example.yml  jvm.options  log4j2.properties  roles.yml  users_roles
root@webserver:/etc/elasticsearch#
```

Figure 4. 14 Elasticsearch configuration

```
GNU nano 7.2 elasticsearch.yml
# Add custom attributes to the node:
#
#node.attr.rack: r1
#
# ----- Paths -----
#
# Path to directory where to store the data (separate multiple locations by comma):
#
path.data: /var/lib/elasticsearch
#
# Path to log files:
#
path.logs: /var/log/elasticsearch
#
# ----- Memory -----
#
# Lock the memory on startup:
#bootstrap.memory_lock: true
#
# Make sure that the heap size is set to about half the memory available
# on the system and that the owner of the process is allowed to use this
# limit.
#
# Elasticsearch performs poorly when the system is swapping the memory.
#
# ----- Network -----
#
# By default Elasticsearch is only accessible on localhost. Set a different
# address here to expose this node on the network:
#
network.host: 192.168.1.102
#
# By default Elasticsearch listens for HTTP traffic on the first free port it
# finds starting at 9200. Set a specific HTTP port here:
#
http.port: 9200
#
# For more information, consult the network module documentation.
#
# ----- Discovery -----
#
#discovery.type: single-node
#
# Pass an initial list of hosts to perform discovery when this node is started:
# The default list of hosts is ["127.0.0.1", "::1"]
```

For better performance the jvm.options is configured in Elasticsearch to run Fastly and smoothly. It increases the memory space for the elasticsearch.

Figure 4. 15 jvm.options

```
## and the min and max should be set to the same value. For
## example, to set the heap to 4 GB, create a new file in the
## jvm.options.d directory containing these lines:
##
## -Xms4g
## -Xmx4g
##
## See https://www.elastic.co/guide/en/elasticsearch/reference/7.17/heap-size.html
## for more information
##
#####
-Xmx512m
-Xmx512m
#####
## Expert settings
#####
## All settings below here are considered expert settings. Do
## not adjust them unless you understand what you are doing. Do

[ Read 101 lines ]
Help Write Out Where Is Cut Execute Location M-U Undo
Exit Read File Replace Paste Justify Go To Line M-E Redo
```

Then, Logstash is installed and configured to access the logs from the web application (Refer Appendix H).

Figure 4. 16 Logstash directory

```

root@webserver:/etc/logstash# ls
conf.d  jvm.options  log4j2.properties  logstash-sample.conf  logstash.yml  pipelines.yml  startup.options
root@webserver:/etc/logstash# cd conf.d
root@webserver:/etc/logstash/conf.d# ls
webapp-log.conf
root@webserver:/etc/logstash/conf.d# _

```

Figure 4. 17 Logstash configuration file

```

GNU nano 7.2
input {
  file {
    path => "/var/www/webapp/logs/access.log"
    start_position => "beginning"
    sincedb_path => "/dev/null"
  }
  file {
    path => "/var/www/webapp/logs/error.log"
    start_position => "beginning"
    sincedb_path => "/dev/null"
  }
}

filter {
  grok {
    match => { "message" => "%{COMMONAPACHELOG}" }
  }
  date {
    match => [ "timestamp" , "dd/MMM/yyyy:HH:mm:ss Z" ]
  }
}

output {
  elasticsearch {
    hosts => ["http://192.168.1.102:9200"]
    index => "webapp-logs-%{+YYYY.MM.dd}"
  }
  stdout { codec => rubydebug }
}

```

Finally, Kibana is installed and the Kibana configuration file is configured (Refer Appendix I).

Figure 4. 18 Kibana directory

```

root@webserver:~# cd /etc/kibana
root@webserver:/etc/kibana# ls
kibana.keystore  kibana.yml  node.options
root@webserver:/etc/kibana# _

```

Figure 4. 19 Kibana configuration file

```

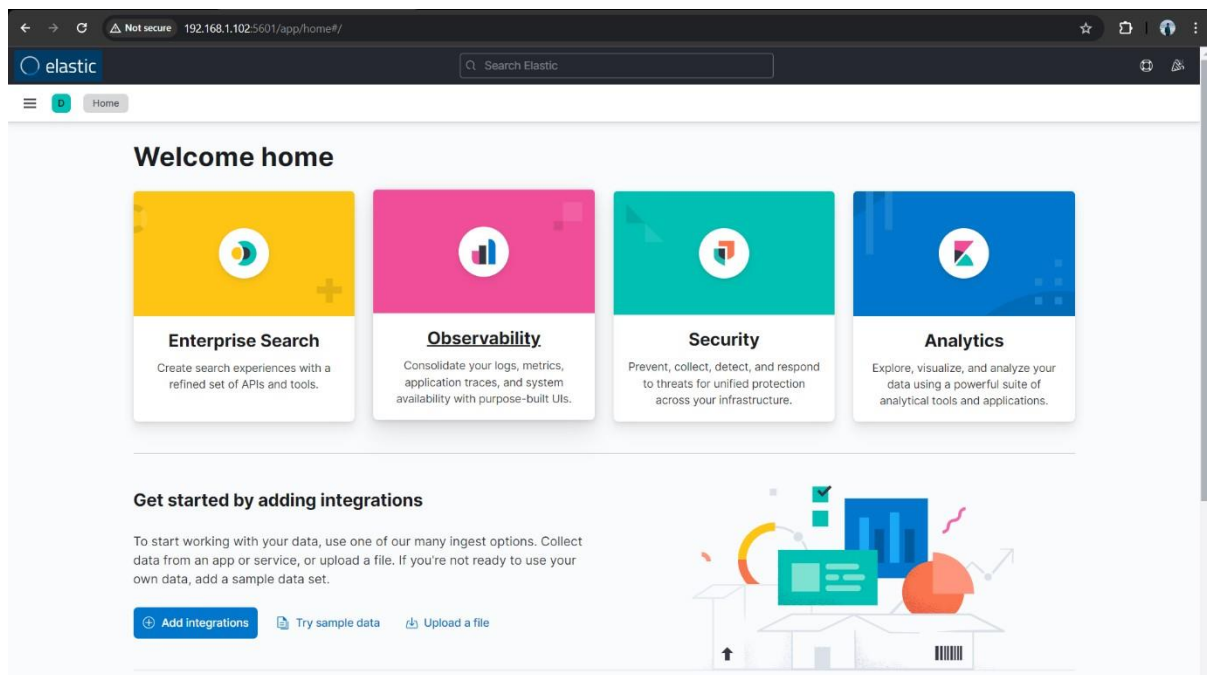
GNU nano 7.2 kibana.yml
# Kibana is served by a back end server. This setting specifies the port to use.
#server.port: 5601

# Specifies the address to which the Kibana server will bind. IP addresses and host names are both valid values.
# The default is 'localhost', which usually means remote machines will not be able to connect.
# To allow connections from remote users, set this parameter to a non-loopback address.
server.port: 5601
server.host: "192.168.1.102"
elasticsearch.hosts: ["http://192.168.1.102:9200"]
#xpack.security.enabled: true
# Enables you to specify a path to mount Kibana at if you are running behind a proxy.
# Use the 'server.rewriteBasePath' setting to tell Kibana if it should remove the basePath
# from requests it receives, and to prevent a deprecation warning at startup.
# This setting cannot end in a slash.
#server.basePath: ""

```

After Elasticsearch, Logstash and Kibana is configured and then each service is started one by one. The web interface of ELK (192.168.1.102:5601) is accessible from the host.

Figure 4. 20 ELK web interface



4.6 Setting Suricata and Filebeat

The Suricata and filebeat is installed like the same way Elasticsearch is installed, creating a repository and then it is installed. After installing Suricata, its configuration file is modified to collect the traffic logs of server's network and to save them in file eve.json which is in var directory (Refer Appendix K).

Figure 4. 21 Suricata directory

```
root@webserver:/etc/suricata# ls
classification.config reference.config rules suricata.yaml threshold.config
root@webserver:/etc/suricata#
```

Figure 4. 22 Suricata configuration file

```
# Linux high speed capture support
af-packet:
- interface: enp0s3
  # Number of receive threads. "auto" uses the number of cores
  #threads: auto
  # Default clusterid. AF_PACKET will load balance packets based on flow.
  cluster-id: 99
```

Figure 4. 23 Suricata in var

```
root@webserver:/var/log/suricata# ls
eve.json fast.log stats.log suricata.log
root@webserver:/var/log/suricata#
```

Then, Filebeat is installed, and its configuration file is configured to ship logs from Suricata's log saved file (eve.json) to Elasticsearch file directly (Refer Appendix L).

Figure 4. 24 Filebeat directory

```
root@webserver:~# cd /etc/filebeat
root@webserver:/etc/filebeat# ls
fields.yml filebeat.reference.yml filebeat.yml modules.d
root@webserver:/etc/filebeat#
```

Figure 4. 25 Filebeat configuration file

```
GNU nano 7.2 filebeat.yml
##### Filebeat Configuration Example #####

# This file is an example configuration file highlighting only the most common
# options. The filebeat.reference.yml file from the same directory contains all the
# supported options with more comments. You can use it as a reference.
#
# You can find the full configuration reference here:
# https://www.elastic.co/guide/en/beats/filebeat/index.html
#
# For more available modules and options, please see the filebeat.reference.yml sample
# configuration file.

# ===== Filebeat inputs =====

filebeat.inputs:

# Each - is an input. Most options can be set at the input level, so
# you can use different inputs for various configurations.
# Below are the input specific configurations.

# filestream is an input for collecting log messages from files.
- type: log
  enabled: true

  # Unique ID among all inputs, an ID is required.
  #id: my-filestream-id

  # Change to true to enable this input configuration.
  #enabled: false

  # Paths that should be crawled and fetched. Glob based paths.
  paths:
    - /var/log/suricata/eve.json
  # json.keys_under_root: true
  # json.add_error_key: true
  # - /var/www/webapp/logs/error.log
  # - c:\programdata\elasticsearch\logs\*

# Starting with Beats version 6.0.0, the dashboards are loaded via the Kibana API.
# This requires a Kibana endpoint configuration.
setup.kibana:
  host: "http://192.168.1.102:5601"

setup.ilm.enabled: auto
setup.ilm.overwrite: true

# ===== Elasticsearch Output =====
output.elasticsearch:
  # Array of hosts to connect to.
  hosts: ["http://192.168.1.102:9200"]
  #index: 'suricata-logs-%{+yyyy.MM.dd}'

# Sets log level. The default log level is info.
# Available log levels are: error, warning, info, debug
logging.level: info
logging.to_files: true
logging.files:
  path: /var/log/filebeat
  name: filebeat.log
  keepfiles: 7
  permissions: 0644

# At debug level, you can selectively enable logging only for some components.
# To enable all selectors use ["*"]. Examples of other selectors are "beat",
# "publisher", "service".
#logging.selectors: ["*"]
```

After Suricata and Filebeat is configured and then each service is started.

4.7 Setting Pfsense Rules

After setting monitoring system in ubuntu, port forwarding, and rules are set up from the Pfsense web interface. The figures shown below is the rules and setting made in the pfsense.

Figure 4. 26 Pfsense port forwarding

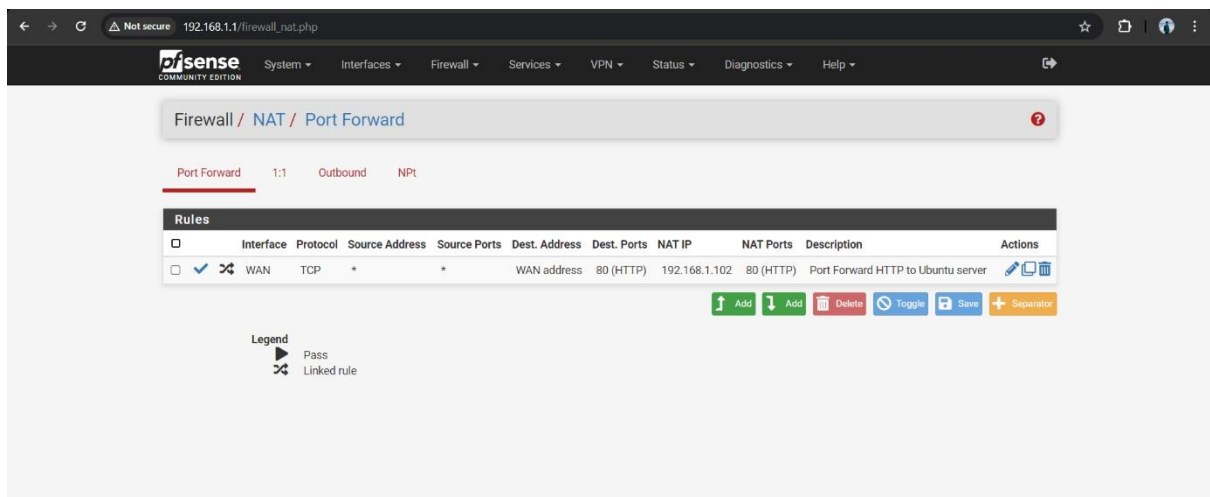
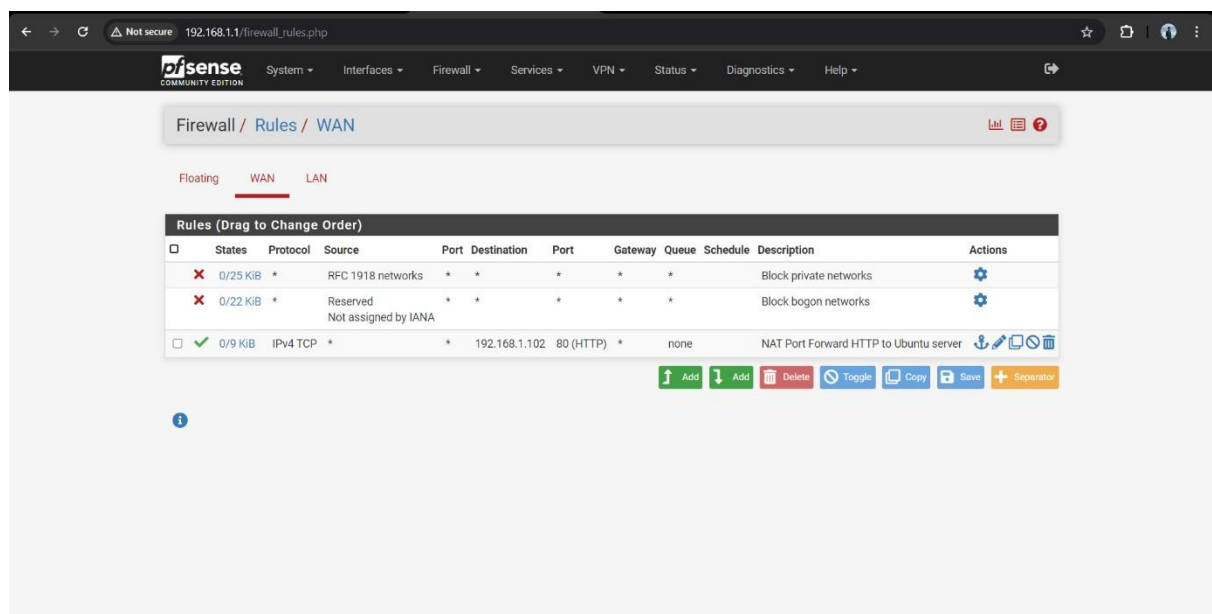


Figure 4. 27 Pfsense rule



4.8 Setting Router rules

The router is Virgin media hub 3.0. Router is set up to forward the WAN to the public IP address of the host device to receive and sent requests and data. The Router interface and the Port forwarding rules is shown below.

Figure 4. 28 Router interface

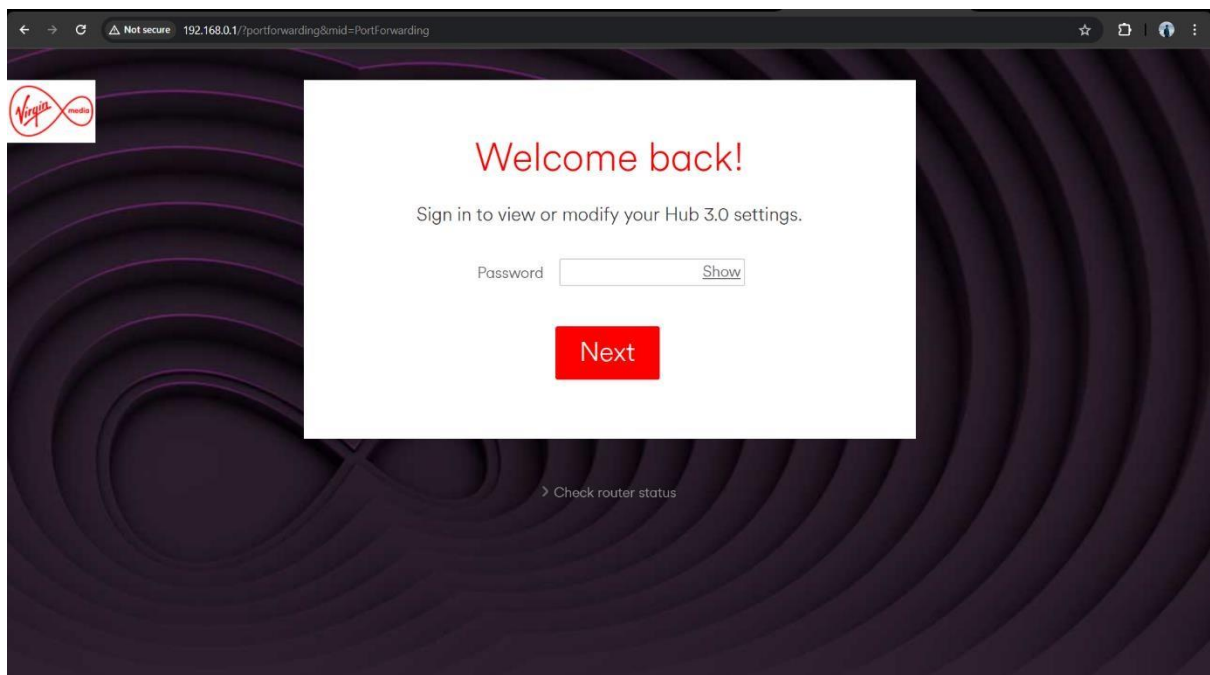
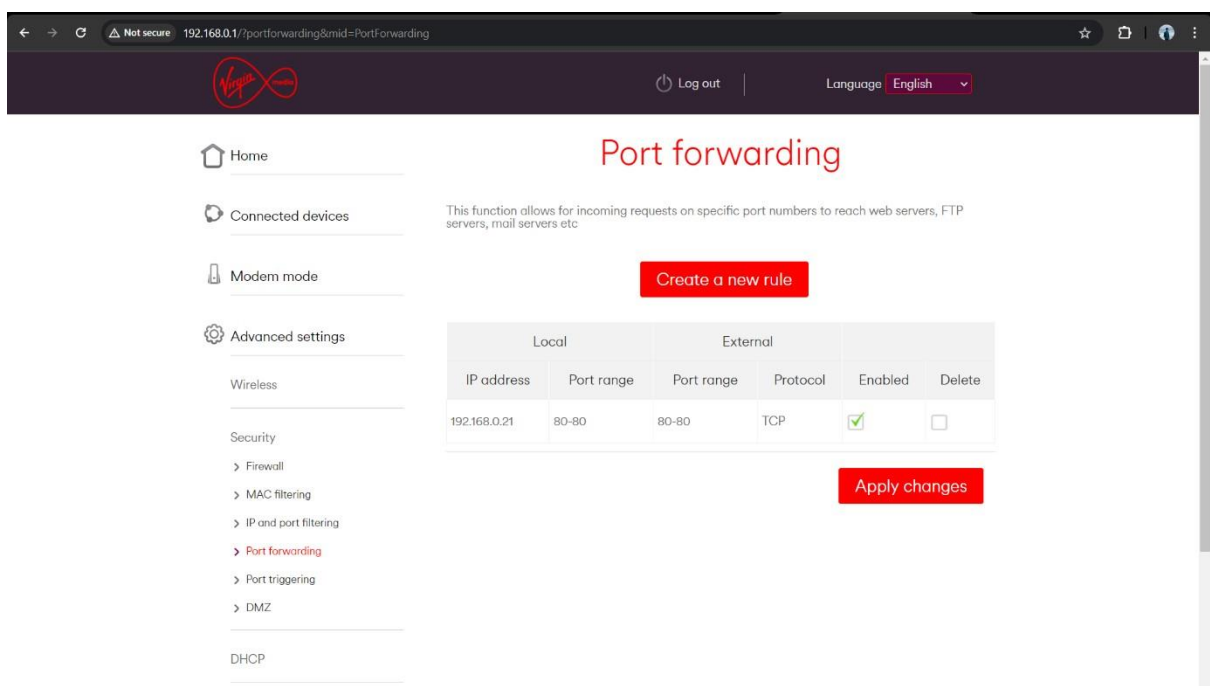


Figure 4. 29 Router port forwarding



4.9 Setting DNS

DNS is bought from GoDaddy, and domain setting is set up. The domain name is “theanugrah.com”. The setting is shown below.

Figure 4. 30 DNS settings

CHAPTER 5

5. Accuracy and Testing

5.1 Web application monitoring

To ensure web application monitoring is working properly, the status of monitoring systems and data collection need to be checked.

5.1.1 Web service status

To guarantee that the web application can be hosted without any issues, the Apache2 service's status is checked. This is an essential step in making sure the web application runs smoothly.

In the below figure, it is shown the apache2 configuration is correct and is running without any issue (Refer appendix B).

Figure 5. 1 Apache2 status

```

root@webserver:~# systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
   Active: active (running) since Wed 2024-07-31 13:51:10 UTC; 3h 28min ago
     Docs: https://httpd.apache.org/docs/2.4/
   Process: 4496 ExecReload=/usr/sbin/apachectl graceful (code=exited, status=0/SUCCESS)
   Main PID: 1974 (apache2)
     Tasks: 55 (limit: 4614)
    Memory: 8.5M (peak: 12.1M)
       CPU: 1.443s
   CGroup: /system.slice/apache2.service
           └─1974 /usr/sbin/apache2 -k start
             └─4502 /usr/sbin/apache2 -k start
               └─4503 /usr/sbin/apache2 -k start

Jul 31 13:51:08 webserver systemd[1]: Starting apache2.service - The Apache HTTP Server...
Jul 31 13:51:10 webserver apachectl[1964]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1. Set the 'ServerName'
Jul 31 13:51:10 webserver systemd[1]: Started apache2.service - The Apache HTTP Server.
Jul 31 15:42:46 webserver systemd[1]: Reloading apache2.service - The Apache HTTP Server...
Jul 31 15:42:46 webserver apachectl[4499]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1. Set the 'ServerName'
Jul 31 15:42:46 webserver systemd[1]: Reloaded apache2.service - The Apache HTTP Server.
lines 1-20/20 (END)

```

It's important to frequently verify that Elasticsearch, Kibana, and Logstash are operating as planned to ensure that the monitoring system is correctly and functioning as planned.

The below figure shows that the Elasticsearch is enabled and active. This shows that it has no error and runs smoothly (Refer appendix G).

Figure 5. 2 Elasticsearch status

```

root@webserver:~# systemctl status elasticsearch
● elasticsearch.service - Elasticsearch
   Loaded: loaded (/usr/lib/systemd/system/elasticsearch.service; enabled; preset: enabled)
   Active: active (running) since Wed 2024-07-31 13:53:22 UTC; 3h 24min ago
     Docs: https://www.elastic.co
   Main PID: 1927 (java)
     Tasks: 68 (limit: 4614)
    Memory: 986.9M (peak: 1.0G)
       CPU: 31min 10.044s
   CGroup: /system.slice/elasticsearch.service
           └─1927 /usr/share/elasticsearch/jdk/bin/java -Xshare:auto -Des.networkaddress.cache.ttl=60 -Des.networkaddress.cache.negative.ttl=10 -XX:+AlwaysPrefer
              3030 /usr/share/elasticsearch/modules/x-pack-m1/platform/linux-x86_64/bin/controller

Jul 31 13:51:00 webserver systemd[1]: Starting elasticsearch.service - Elasticsearch...
Jul 31 13:51:53 webserver systemd-entrypoint[1927]: Jul 31, 2024 1:51:53 PM sun.util.locale.provider.LocaleProviderAdapter <clinit>
Jul 31 13:51:53 webserver systemd-entrypoint[1927]: WARNING: COMPAT locale provider will be removed in a future release
Jul 31 13:53:22 webserver systemd[1]: Started elasticsearch.service - Elasticsearch.
lines 1-16/16 (END)

```

Logstash is running smoothly and enabled (Refer appendix H), it is shown in the figure below.

Figure 5. 3 Logstash status

```

root@webserver:~# systemctl status logstash
● logstash.service - logstash
   Loaded: loaded (/etc/systemd/system/logstash.service; enabled; preset: enabled)
   Active: active (running) since Wed 2024-07-31 14:10:32 UTC; 3h 7min ago
     Main PID: 3582 (java)
       Tasks: 35 (limit: 4614)
      Memory: 532.6M (peak: 551.1M)
         CPU: 2min 45.866s
      CGroup: /system.slice/logstash.service
              └─3582 /usr/share/logstash/jdk/bin/java -Xms1g -Xmx1g -XX:+UseConcMarkSweepGC -XX:CMSInitiatingOccupancyFraction=75 -XX:+UseCMSInitiatingOccupancy
Jul 31 16:29:05 webserver logstash[3582]: "timestamp" => "31/Jul/2024:16:29:04 +0000",
Jul 31 16:29:05 webserver logstash[3582]: "version" => "1",
Jul 31 16:29:05 webserver logstash[3582]: "clientip" => "173.44.141.11",
Jul 31 16:29:05 webserver logstash[3582]: "rawrequest" => "\\x16\\x03\\x01",
Jul 31 16:29:05 webserver logstash[3582]: "auth" => "",
Jul 31 16:29:05 webserver logstash[3582]: "message" => "173.44.141.11 - - [31/Jul/2024:16:29:04 +0000] \\x16\\x03\\x01\" 400 488 \"-\" \"-\" \"-\",
Jul 31 16:29:05 webserver logstash[3582]: "ident" => "logstash[3582]",
Jul 31 16:29:05 webserver logstash[3582]: "host" => "webserver",
Jul 31 16:29:05 webserver logstash[3582]: "bytes" => "488"
Jul 31 16:29:05 webserver logstash[3582]: }
lines 1-20/20 (END)

```

In the below figure, Kibana is also running smoothly and active (Refer appendix I).

Figure 5. 4 Kibana status

```

root@webserver:~# systemctl status kibana
• kibana.service - Kibana
   Loaded: loaded (/etc/systemd/system/kibana.service; enabled; preset: enabled)
   Active: active (running) since Wed 2024-07-31 13:51:08 UTC; 3h 26min ago
     Docs: https://www.elastic.co
   Main PID: 1929 (node)
    Tasks: 11 (limit: 4614)
   Memory: 556.3M (peak: 820.7M)
      CPU: 4min 7.340s
   CGroup: /system.slice/kibana.service
           └─1929 /usr/share/kibana/bin/../node/bin/node /usr/share/kibana/bin/../src/cli/dist --logging.dest=/var/log/kibana/kibana.log --pid.file=/run/kibana
Jul 31 13:51:08 webserver systemd[1]: Started kibana.service - Kibana.
Jul 31 13:51:15 webserver kibana[1929]: Kibana is currently running with legacy OpenSSL providers enabled! For details and instructions on how to disable see https://www.elastic.co/guide/en/kibana/current/openssl.html
lines 1-13/13 (END)

```

5.1.2 Collecting web logs

It is necessary to gather the logs so that the system can operate efficiently and deliver correct monitoring data. For best results, it's crucial to routinely check web application logs, including access and error logs (Refer appendix C).

The below figure shows that the access logs are being saved in the correct file within the server itself.

Figure 5. 5 Access logs

```

192.168.1.100 - - [31/Jul/2024:13:54:50 +0000] "GET / HTTP/1.1" 200 703 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/122.0.0.0 Safari/537.36"
192.168.1.100 - - [31/Jul/2024:13:54:51 +0000] "GET / HTTP/1.1" 200 703 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/122.0.0.0 Safari/537.36"
192.168.1.100 - - [31/Jul/2024:13:55:42 +0000] "-" 400 0 "-" "-"
192.168.1.100 - - [31/Jul/2024:13:57:06 +0000] "GET / HTTP/1.1" 200 704 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/122.0.0.0 Safari/537.36"
192.168.1.100 - [31/Jul/2024:13:57:09 +0000] "GET /about.html HTTP/1.1" 200 706 "http://192.168.1.102/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/122.0.0.0 Safari/537.36"
192.168.1.100 - [31/Jul/2024:13:57:11 +0000] "GET /index.html HTTP/1.1" 200 703 "http://192.168.1.102/about.html" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/122.0.0.0 Safari/537.36"
192.168.1.100 - [31/Jul/2024:13:57:15 +0000] "GET /contact.html HTTP/1.1" 200 697 "http://192.168.1.102/index.html" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/122.0.0.0 Safari/537.36"
192.168.1.100 - [31/Jul/2024:13:57:58 +0000] "-" 400 0 "-" "-"
192.168.1.100 - [31/Jul/2024:14:51:53 +0000] "GET /index.html HTTP/1.1" 200 704 "http://192.168.1.102/contact.html" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/122.0.0.0 Safari/537.36"
192.168.1.100 - [31/Jul/2024:14:51:53 +0000] "GET /index.html HTTP/1.1" 200 703 "http://192.168.1.102/contact.html" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/122.0.0.0 Safari/537.36"
192.168.1.100 - [31/Jul/2024:14:51:54 +0000] "GET /index.html HTTP/1.1" 200 703 "http://192.168.1.102/contact.html" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/122.0.0.0 Safari/537.36"
192.168.1.100 - [31/Jul/2024:14:52:45 +0000] "-" 400 0 "-" "-"
192.168.1.100 - [31/Jul/2024:15:42:56 +0000] "GET /index.html HTTP/1.1" 200 704 "http://192.168.1.102/contact.html" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/122.0.0.0 Safari/537.36"
192.168.1.100 - [31/Jul/2024:15:42:56 +0000] "GET /index.html HTTP/1.1" 200 703 "http://192.168.1.102/contact.html" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/122.0.0.0 Safari/537.36"
192.168.1.100 - [31/Jul/2024:15:43:48 +0000] "-" 400 0 "-" "-"
192.168.1.100 - [31/Jul/2024:16:20:16 +0000] "GET / HTTP/1.1" 200 1025 "-" "-"
192.168.1.100 - [31/Jul/2024:16:22:00 +0000] "HEAD /wordpress HTTP/1.1" 404 140 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/74.0.3729.169 Safari/537.36"
207.140.11.64 - [31/Jul/2024:16:22:00 +0000] "HEAD / HTTP/1.1" 200 252 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/74.0.3729.169 Safari/537.36"
207.140.11.64 - [31/Jul/2024:16:22:09 +0000] "HEAD /wp HTTP/1.1" 404 140 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/74.0.3729.169 Safari/537.36"
207.140.11.64 - [31/Jul/2024:16:22:09 +0000] "HEAD /bc HTTP/1.1" 404 140 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/74.0.3729.169 Safari/537.36"
207.140.11.64 - [31/Jul/2024:16:22:09 +0000] "HEAD /db HTTP/1.1" 404 140 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/74.0.3729.169 Safari/537.36"
207.140.11.64 - [31/Jul/2024:16:22:09 +0000] "HEAD /backup HTTP/1.1" 404 140 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/74.0.3729.169 Safari/537.36"
207.140.11.64 - [31/Jul/2024:16:22:09 +0000] "HEAD /old HTTP/1.1" 404 140 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/74.0.3729.169 Safari/537.36"
207.140.11.64 - [31/Jul/2024:16:22:09 +0000] "HEAD /new HTTP/1.1" 404 140 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/74.0.3729.169 Safari/537.36"
207.140.11.64 - [31/Jul/2024:16:22:09 +0000] "HEAD /main HTTP/1.1" 404 140 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/74.0.3729.169 Safari/537.36"
207.140.11.64 - [31/Jul/2024:16:22:09 +0000] "HEAD /home HTTP/1.1" 404 140 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/74.0.3729.169 Safari/537.36"
45.141.215.232 - [31/Jul/2024:16:27:22 +0000] "GET / HTTP/1.1" 200 667 "-" "Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:76.0) Gecko/20100101 Firefox/76.0"
173.48.141.11 - [31/Jul/2024:16:29:04 +0000] "GET /wp-content/themes/2016/404 400 "-" "-"
root@webserver:~# cat /var/www/webapp/logs#

```

The error logs are also being saved in the correct file in the server, that is shown in the below figure.

Figure 5. 6 Error logs

```

[Mon Jul 08 13:35:51.003959 2024] [autoindex:error] [pid 4261:tfd 123138966016440] [client 192.168.1.100:9588] AH01276: Cannot serve directory /var/www/webapp/p
public_html/: No matching DirectoryIndex (index.html,index.cgi,index.pl,index.php,index.shtml,index.htm) found, and server-generated directory index forbidden by
Options directive
[Mon Jul 08 13:35:51.007288 2024] [autoindex:error] [pid 4261:tfd 123138966030688] [client 192.168.1.100:9588] AH01276: Cannot serve directory /var/www/webapp/p
public_html/: No matching DirectoryIndex (index.html,index.cgi,index.pl,index.php,index.shtml,index.htm) found, and server-generated directory index forbidden by
Options directive
[Mon Jul 08 13:35:50.998136 2024] [autoindex:error] [pid 4262:tfd 123138967787868] [client 192.168.1.100:9588] AH01276: Cannot serve directory /var/www/webapp/p
public_html/: No matching DirectoryIndex (index.html,index.cgi,index.pl,index.php,index.shtml,index.htm) found, and server-generated directory index forbidden by
Options directive
[Mon Jul 08 13:35:17.541711 2024] [autoindex:error] [pid 4423:tfd 139750717523640] [client 192.168.1.100:9605] AH01276: Cannot serve directory /var/www/webapp/p
public_html/: No matching DirectoryIndex (index.html,index.cgi,index.pl,index.php,index.shtml,index.htm) found, and server-generated directory index forbidden by
Options directive
[Mon Jul 08 13:35:17.701159 2024] [autoindex:error] [pid 4423:tfd 139750707037888] [client 192.168.1.100:9605] AH01276: Cannot serve directory /var/www/webapp/p
public_html/: No matching DirectoryIndex (index.html,index.cgi,index.pl,index.php,index.shtml,index.htm) found, and server-generated directory index forbidden by
Options directive
[Mon Jul 08 13:35:17.817643 2024] [autoindex:error] [pid 4423:tfd 139750696552128] [client 192.168.1.100:9605] AH01276: Cannot serve directory /var/www/webapp/p
public_html/: No matching DirectoryIndex (index.html,index.cgi,index.pl,index.php,index.shtml,index.htm) found, and server-generated directory index forbidden by
Options directive
[Mon Jul 08 13:35:20.484090 2024] [autoindex:error] [pid 4423:tfd 139750686066368] [client 192.168.1.100:9605] AH01276: Cannot serve directory /var/www/webapp/p
public_html/: No matching DirectoryIndex (index.html,index.cgi,index.pl,index.php,index.shtml,index.htm) found, and server-generated directory index forbidden by
Options directive
[Mon Jul 08 13:43:10.416221 2024] [autoindex:error] [pid 4568:tfd 139750717523640] [client 192.168.1.100:9624] AH01276: Cannot serve directory /var/www/webapp/p
public_html/: No matching DirectoryIndex (index.html,index.cgi,index.pl,index.php,index.shtml,index.htm) found, and server-generated directory index forbidden by
Options directive
[Mon Jul 08 13:43:10.457621 2024] [autoindex:error] [pid 4568:tfd 139750707037888] [client 192.168.1.100:9624] AH01276: Cannot serve directory /var/www/webapp/p
public_html/: No matching DirectoryIndex (index.html,index.cgi,index.pl,index.php,index.shtml,index.htm) found, and server-generated directory index forbidden by
Options directive
[Mon Jul 08 13:43:33.336986 2024] [autoindex:error] [pid 4567:tfd 139750717523640] [client 192.168.1.100:9625] AH01276: Cannot serve directory /var/www/webapp/p
public_html/: No matching DirectoryIndex (index.html,index.cgi,index.pl,index.php,index.shtml,index.htm) found, and server-generated directory index forbidden by
Options directive
[Mon Jul 08 13:43:30.389232 2024] [autoindex:error] [pid 4568:tfd 139750686066368] [client 192.168.1.100:9633] AH01276: Cannot serve directory /var/www/webapp/p
public_html/: No matching DirectoryIndex (index.html,index.cgi,index.pl,index.php,index.shtml,index.htm) found, and server-generated directory index forbidden by
Options directive
[Mon Jul 08 13:43:39.854039 2024] [autoindex:error] [pid 4568:tfd 139750675500608] [client 192.168.1.100:9633] AH01276: Cannot serve directory /var/www/webapp/p
public_html/: No matching DirectoryIndex (index.html,index.cgi,index.pl,index.php,index.shtml,index.htm) found, and server-generated directory index forbidden by
Options directive
[Mon Jul 08 13:47:36.280653 2024] [autoindex:error] [pid 4735:tfd 139750717523640] [client 192.168.1.100:9653] AH01276: Cannot serve directory /var/www/webapp/p
public_html/: No matching DirectoryIndex (index.html,index.cgi,index.pl,index.php,index.shtml,index.htm) found, and server-generated directory index forbidden by
Options directive
[Mon Jul 08 13:47:36.628010 2024] [autoindex:error] [pid 4735:tfd 139750707037888] [client 192.168.1.100:9653] AH01276: Cannot serve directory /var/www/webapp/p
public_html/: No matching DirectoryIndex (index.html,index.cgi,index.pl,index.php,index.shtml,index.htm) found, and server-generated directory index forbidden by
Options directive
[Mon Jul 08 13:47:43.342611 2024] [autoindex:error] [pid 4735:tfd 139750696552128] [client 192.168.1.100:9653] AH01276: Cannot serve directory /var/www/webapp/p
public_html/: No matching DirectoryIndex (index.html,index.cgi,index.pl,index.php,index.shtml,index.htm) found, and server-generated directory index forbidden by
Options directive
[Mon Jul 08 13:50:57.027635 2024] [autoindex:error] [pid 4897:tfd 139750644123328] [client 192.168.1.100:9668] AH01276: Cannot serve directory /var/www/webapp/p
public_html/: No matching DirectoryIndex (index.html,index.cgi,index.pl,index.php,index.shtml,index.htm) found, and server-generated directory index forbidden by
Options directive
error log message
root@webserver:/var/www/webapp/logs#

```

5.2 Network monitoring

Regularly checking the state of the monitoring system and data gathering procedures is important for ensuring the correct operation of network monitoring.

5.2.1 Network service status

To provide accurate network monitoring, the functionality of Filebeat and Suricata is checked.

The below figure shows that the Suricata is successfully running in the server and active (Refer appendix K).

Figure 5. 7 Suricata status

```

root@webserver:~# systemctl status suricata
● suricata.service - Suricata IDS/IDP daemon
   Loaded: loaded (/usr/lib/systemd/system/suricata.service; enabled; preset: enabled)
   Active: active (running) since Wed 2024-07-31 13:51:12 UTC; 3h 27min ago
     Main PID: 2053 (Suricata-Main)
        Tasks: 7 (limit: 4614)
      Memory: 421.5M (peak: 437.1M)
         CPU: 3min 6.902s
      CGroup: /system.slice/suricata.service
              └─2053 /usr/bin/suricata -D --af-packet -c /etc/suricata/suricata.yaml --pidfile /run/suricata.pid

Jul 31 13:51:09 webserver systemd[1]: Starting suricata.service - Suricata IDS/IDP daemon...
Jul 31 13:51:11 webserver suricata[1935]: i: suricata: This is Suricata version 7.0.3 RELEASE running in SYSTEM mode
Jul 31 13:51:12 webserver systemd[1]: Started suricata.service - Suricata IDS/IDP daemon.
root@webserver:~#

```

In the below figure, the Filebeat is running properly and active (Refer appendix L).

Figure 5. 8 Filebeat status


```

root@webserver:~# systemctl status filebeat
* filebeat.service - Filebeat sends log files to Logstash or directly to Elasticsearch.
   Loaded: loaded (/usr/lib/systemd/system/filebeat.service; enabled; preset: enabled)
   Active: active (running) since Wed 2024-07-31 13:51:08 UTC; 3h 27min ago
     Docs: https://www.elastic.co/beats/filebeat
    Main PID: 1928 (filebeat)
      Tasks: 8 (limit: 4614)
     Memory: 243.3M (peak: 588.2M)
        CPU: 32.131s
    CGroup: /system.slice/filebeat.service
            └─1928 /usr/share/filebeat/bin/filebeat --environment systemd -c /etc/filebeat/filebeat.yml --path.home /usr/share/filebeat --path.config /etc/fil
Jul 31 13:51:08 webserver systemd[1]: Started filebeat.service - Filebeat sends log files to Logstash or directly to Elasticsearch.
lines 1-12/12 (END)

```

5.2.2 Collecting network logs

Examining the filebeat log and the eve.json file is important for guaranteeing correct data transmission and gathering. This will confirm that the information is being accurately gathered and transmitted straight to Elasticsearch through filebeat.

The below figure shows that the network is being monitored and logs are being saved in the correct file eve.json (Refer appendix K).

Figure 5. 9 eve.json

```

GNU nano 7.2          eve.json
"2024-07-18T16:08:39.515364+0000", "flow_id": "2213473309956136", "in_iface": "enp0s3", "event_type": "dns", "src_ip": "192.168.1.102", "src_port": 45347, "dst_ip": "192.168.1.102", "dst_port": 45347, "length": 100, "offset": 0, "capture": "kernel_packets:32024", "kernel_drops": 0, "errors": 0, "attachments": 0, "event_type": "stats", "stats": {"net_line": 44, "capture": "kernel_packets:32024", "kernel_drops": 0, "errors": 0, "attachments": 0}, "event_type": "stats", "stats": {"net_line": 52, "capture": "kernel_packets:32024", "kernel_drops": 0, "errors": 0, "attachments": 0}, "event_type": "stats", "stats": {"net_line": 60, "capture": "kernel_packets:32024", "kernel_drops": 0, "errors": 0, "attachments": 0}, "event_type": "stats", "stats": {"net_line": 68, "capture": "kernel_packets:32024", "kernel_drops": 0, "errors": 0, "attachments": 0}, "event_type": "stats", "stats": {"net_line": 76, "capture": "kernel_packets:32024", "kernel_drops": 0, "errors": 0, "attachments": 0}, "flow_id": "2052232383964900", "in_iface": "enp0s3", "event_type": "flow", "src_ip": "fe80:0000:0000:0000:5baa:30a2:4d29:b000", "dst_ip": "fe80:0000:0000:0000:5baa:30a2:4d29:b000", "length": 100, "offset": 0, "capture": "kernel_packets:32024", "kernel_drops": 0, "errors": 0, "attachments": 0, "event_type": "stats", "stats": {"net_line": 84, "capture": "kernel_packets:32024", "kernel_drops": 0, "errors": 0, "attachments": 0}, "flow_id": "2153636315119471", "in_iface": "enp0s3", "event_type": "flow", "src_ip": "192.168.1.100", "dst_ip": "192.168.1.100", "length": 100, "offset": 0, "capture": "kernel_packets:32024", "kernel_drops": 0, "errors": 0, "attachments": 0, "flow_id": "9792044113683008", "in_iface": "enp0s3", "event_type": "flow", "src_ip": "192.168.1.100", "dst_ip": "192.168.1.100", "length": 100, "offset": 0, "capture": "kernel_packets:32024", "kernel_drops": 0, "errors": 0, "attachments": 0, "event_type": "stats", "stats": {"net_line": 92, "capture": "kernel_packets:32024", "kernel_drops": 0, "errors": 0, "attachments": 0}, "flow_id": "900342750740888", "in_iface": "enp0s3", "event_type": "flow", "src_ip": "192.168.1.100", "dst_ip": "192.168.1.100", "length": 100, "offset": 0, "capture": "kernel_packets:32024", "kernel_drops": 0, "errors": 0, "attachments": 0, "flow_id": "216347325793297", "in_iface": "enp0s3", "event_type": "flow", "src_ip": "fe80:0000:0000:0000:5baa:30a2:4d29:b000", "dst_ip": "fe80:0000:0000:0000:5baa:30a2:4d29:b000", "length": 100, "offset": 0, "capture": "kernel_packets:32024", "kernel_drops": 0, "errors": 0, "attachments": 0, "event_type": "stats", "stats": {"net_line": 100, "capture": "kernel_packets:32024", "kernel_drops": 0, "errors": 0, "attachments": 0}, "flow_id": "21358809471838470", "in_iface": "enp0s3", "event_type": "flow", "src_ip": "fe80:0000:0000:0000:0000:0000:0000:0000", "dst_ip": "fe80:0000:0000:0000:0000:0000:0000:0000", "length": 100, "offset": 0, "capture": "kernel_packets:32024", "kernel_drops": 0, "errors": 0, "attachments": 0, "flow_id": "2004828741499264", "in_iface": "enp0s3", "event_type": "flow", "src_ip": "192.168.1.102", "dst_ip": "192.168.1.102", "length": 100, "offset": 0, "capture": "kernel_packets:32024", "kernel_drops": 0, "errors": 0, "attachments": 0, "event_type": "stats", "stats": {"net_line": 116, "capture": "kernel_packets:32024", "kernel_drops": 0, "errors": 0, "attachments": 0}, "flow_id": "1435043737671626", "in_iface": "enp0s3", "event_type": "http", "src_ip": "192.168.1.100", "dst_ip": "192.168.1.100", "length": 100, "offset": 0, "capture": "kernel_packets:32024", "kernel_drops": 0, "errors": 0, "attachments": 0, "event_type": "stats", "stats": {"net_line": 132, "capture": "kernel_packets:32024", "kernel_drops": 0, "errors": 0, "attachments": 0}, "flow_id": "1435043737671626", "in_iface": "enp0s3", "event_type": "http", "src_ip": "192.168.1.100", "dst_ip": "192.168.1.100", "length": 100, "offset": 0, "capture": "kernel_packets:32024", "kernel_drops": 0, "errors": 0, "attachments": 0, "event_type": "stats", "stats": {"net_line": 148, "capture": "kernel_packets:32024", "kernel_drops": 0, "errors": 0, "attachments": 0}, "flow_id": "1435043737671626", "in_iface": "enp0s3", "event_type": "http", "src_ip": "192.168.1.100", "dst_ip": "192.168.1.100", "length": 100, "offset": 0, "capture": "kernel_packets:32024", "kernel_drops": 0, "errors": 0, "attachments": 0, "event_type": "stats", "stats": {"net_line": 156, "capture": "kernel_packets:32024", "kernel_drops": 0, "errors": 0, "attachments": 0}, "flow_id": "1435043737671626", "in_iface": "enp0s3", "event_type": "http", "src_ip": "192.168.1.100", "dst_ip": "192.168.1.100", "length": 100, "offset": 0, "capture": "kernel_packets:32024", "kernel_drops": 0, "errors": 0, "attachments": 0, "event_type": "stats", "stats": {"net_line": 164, "capture": "kernel_packets:32024", "kernel_drops": 0, "errors": 0, "attachments": 0}, "flow_id": "1435043737671626", "in_iface": "enp0s3", "event_type": "fileinfo", "src_ip": "192.168.1.102", "dst_ip": "192.168.1.102", "length": 100, "offset": 0, "capture": "kernel_packets:32024", "kernel_drops": 0, "errors": 0, "attachments": 0, "event_type": "stats", "stats": {"net_line": 172, "capture": "kernel_packets:32024", "kernel_drops": 0, "errors": 0, "attachments": 0}, "flow_id": "228189099292368", "in_iface": "enp0s3", "event_type": "flow", "src_ip": "fe80:0000:0000:0000:0000:0000:0000:0000", "dst_ip": "fe80:0000:0000:0000:0000:0000:0000:0000", "length": 100, "offset": 0, "capture": "kernel_packets:32024", "kernel_drops": 0, "errors": 0, "attachments": 0, "event_type": "stats", "stats": {"net_line": 188, "capture": "kernel_packets:32024", "kernel_drops": 0, "errors": 0, "attachments": 0}, "flow_id": "988626161377779", "in_iface": "enp0s3", "event_type": "flow", "src_ip": "192.168.1.100", "dst_ip": "192.168.1.100", "length": 100, "offset": 0, "capture": "kernel_packets:32024", "kernel_drops": 0, "errors": 0, "attachments": 0, "event_type": "stats", "stats": {"net_line": 204, "capture": "kernel_packets:32024", "kernel_drops": 0, "errors": 0, "attachments": 0}, "flow_id": "931676522101319", "in_iface": "enp0s3", "event_type": "flow", "src_ip": "192.168.1.100", "dst_ip": "192.168.1.100", "length": 100, "offset": 0, "capture": "kernel_packets:32024", "kernel_drops": 0, "errors": 0, "attachments": 0, "event_type": "stats", "stats": {"net_line": 220, "capture": "kernel_packets:32024", "kernel_drops": 0, "errors": 0, "attachments": 0}, "flow_id": "931676522101319", "in_iface": "enp0s3", "event_type": "flow", "src_ip": "192.168.1.100", "dst_ip": "192.168.1.100", "length": 100, "offset": 0, "capture": "kernel_packets:32024", "kernel_drops": 0, "errors": 0, "attachments": 0, "event_type": "stats", "stats": {"net_line": 228, "capture": "kernel_packets:32024", "kernel_drops": 0, "errors": 0, "attachments": 0}, "flow_id": "931676522101319", "in_iface": "enp0s3", "event_type": "flow", "src_ip": "192.168.1.100", "dst_ip": "192.168.1.100", "length": 100, "offset": 0, "capture": "kernel_packets:32024", "kernel_drops": 0, "errors": 0, "attachments": 0, "event_type": "stats", "stats": {"net_line": 236, "capture": "kernel_packets:32024", "kernel_drops": 0, "errors": 0, "attachments": 0}, "flow_id": "931676522101319", "in_iface": "enp0s3", "event_type": "flow", "src_ip": "192.168.1.100", "dst_ip": "192.168.1.100", "length": 100, "offset": 0, "capture": "kernel_packets:32024", "kernel_drops": 0, "errors": 0, "attachments": 0, "event_type": "stats", "stats": {"net_line": 244, "capture": "kernel_packets:32024", "kernel_drops": 0, "errors": 0, "attachments": 0}, "flow_id": "931676522101319", "in_iface": "enp0s3", "event_type": "flow", "src_ip": "192.168.1.100", "dst_ip": "192.168.1.100", "length": 100, "offset": 0, "capture": "kernel_packets:32024", "kernel_drops": 0, "errors": 0, "attachments": 0, "event_type": "stats", "stats": {"net_line": 252, "capture": "kernel_packets:32024", "kernel_drops": 0, "errors": 0, "attachments": 0}

```

Filebeat is successfully collecting data from the Suricata eve.json file and transferring it to the Elasticsearch (Refer appendix L), it is shown in the figure below.

Figure 5. 10 Filebeat log

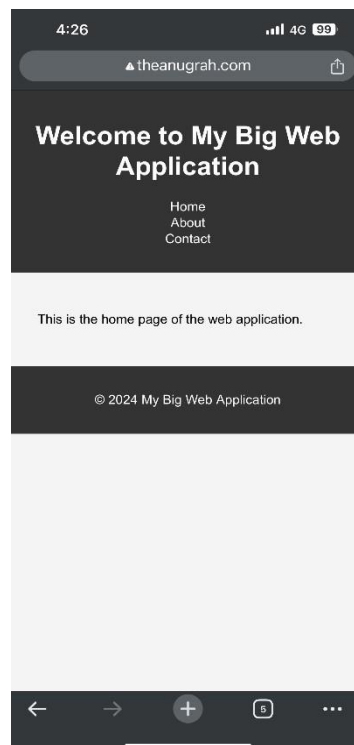
```
GNU nano 7.2 filebeat.log
2024-07-31T14:47:26.049Z INFO instance/beat.go:698 Home path: [/usr/share/filebeat] Config path: [/etc/filebeat] Data path: [/var/lib/filebeat] Log
2024-07-31T14:47:26.049Z INFO instance/beat.go:706 Beat ID: 0d912282-ac63-43ad-8e40-c799b2f4c366
2024-07-31T14:47:26.050Z INFO [beat] instance/beat.go:1052 Beat info {"system_info": {"beat": {"path": {"config": "/etc/filebeat", "data": "
2024-07-31T14:47:26.050Z INFO [beat] instance/beat.go:1061 Build info {"system_info": {"build": {"commit": "51edb8fa613407bf41b006b4415557707
2024-07-31T14:47:26.050Z INFO [beat] instance/beat.go:1064 Go runtime info {"system_info": {"go": {"os": "linux", "arch": "amd64", "max_procs": 1, "vers
2024-07-31T14:47:26.050Z INFO [beat] instance/beat.go:1070 Host info {"system_info": {"host": {"architecture": "x86_64", "boot_time": "2024-07-
2024-07-31T14:47:26.050Z INFO [beat] instance/beat.go:1099 Process info {"system_info": {"process": {"capabilities": {"inheritable": null, "permi
2024-07-31T14:47:26.050Z INFO instance/beat.go:292 Setup Beat: filebeat; Version: 7.17.22
2024-07-31T14:47:26.050Z INFO [index-management] idmgmt/std.go:184 Set output.elasticsearch.index to 'filebeat-7.17.22' as ILM is enabled.
2024-07-31T14:47:26.051Z INFO [esclientleg] eslegclient/connection.go:105 elasticsearch url: http://192.168.1.102:9200
2024-07-31T14:47:26.051Z INFO [publisher] pipeline/module.go:113 Beat name: webserver
2024-07-31T14:47:26.052Z INFO [esclientleg] eslegclient/connection.go:105 elasticsearch url: http://192.168.1.102:9200
2024-07-31T14:47:26.061Z INFO [esclientleg] eslegclient/connection.go:285 Attempting to connect to Elasticsearch version 7.17.22
2024-07-31T14:47:26.089Z INFO [index-management] idmgmt/std.go:260 Auto ILM enable success.
2024-07-31T14:47:26.293Z INFO [index-management.ilm] ilm/std.go:180 ILM policy filebeat successfully created.
2024-07-31T14:47:26.293Z INFO [index-management] idmgmt/std.go:396 Set setup.template.name to 'filebeat-7.17.22 {now/d}-000001' as ILM is
2024-07-31T14:47:26.293Z INFO [index-management] idmgmt/std.go:401 Set setup.template.pattern to 'filebeat-7.17.22-*' as ILM is enabled.
2024-07-31T14:47:26.293Z INFO [index-management] idmgmt/std.go:435 Set settings.index.lifecycle.rollover.alias in template to {filebeat-7.
2024-07-31T14:47:26.294Z INFO [index-management] idmgmt/std.go:439 Set settings.index.lifecycle.name in template to {filebeat {policy":{
2024-07-31T14:47:26.320Z INFO template/load.go:197 Existing template will be overwritten, as overwrite is enabled.
2024-07-31T14:47:27.854Z INFO template/load.go:131 Try loading template filebeat-7.17.22 to Elasticsearch
2024-07-31T14:47:27.939Z INFO template/load.go:123 Template with name "filebeat-7.17.22" loaded.
2024-07-31T14:47:27.939Z INFO [index-management] idmgmt/std.go:296 Loaded index template.
2024-07-31T14:47:27.956Z INFO [index-management.ilm] ilm/std.go:136 Index Alias filebeat-7.17.22 exists already.
```

5.3 Simulation Result

To verify that the system is operating error-free and produces the expected output, a simulation test is run to make sure everything is working as it should.

Using the DNS, the web application is accessed from an apple mobile device using the internet.

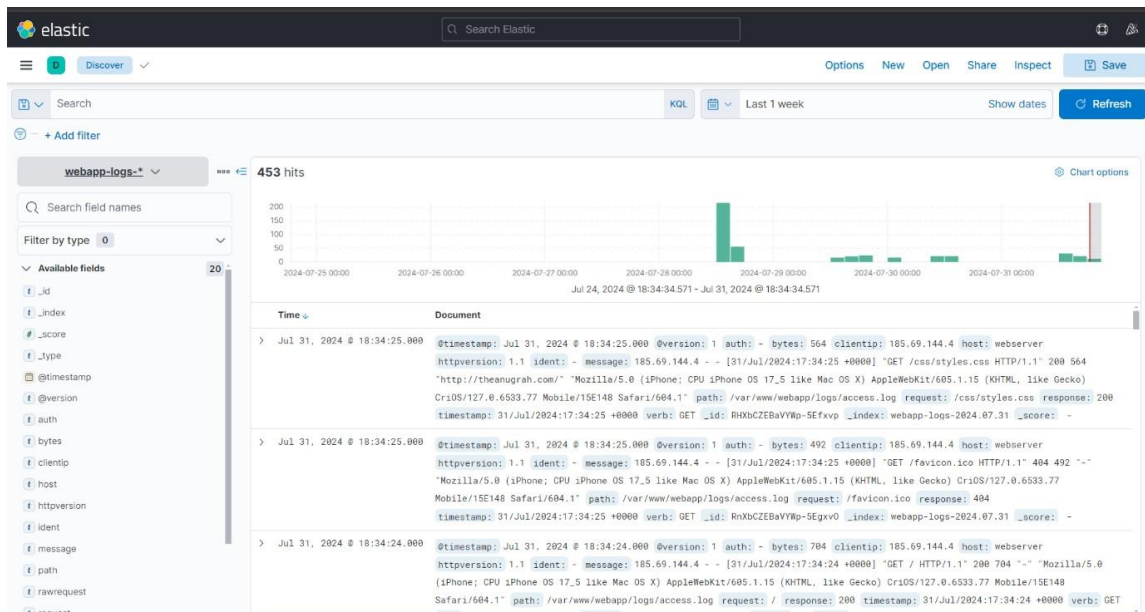
Figure 5. 11 Website simulation test



Through the ELK interface, the logs are visualized from the created index patterns (Refer Appendix L), the IP address of the Apple device accessing the web application is monitored. It can also filter by error response and IP address for finding attack pattern and other purposes.

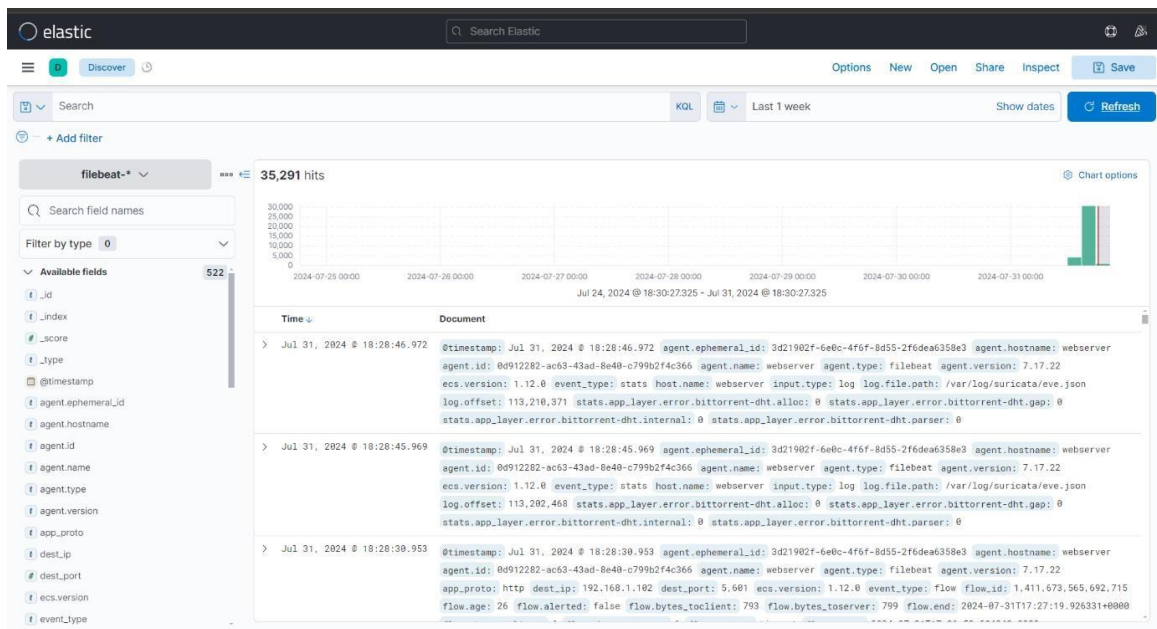
The monitoring simulation results are shown in the below figure.

Figure 5. 12 Simulation result



The ELK interface will capture and show any activity occurring on the server network, which it will be continuously monitoring. The given below figure shows that the result of the network monitoring.

Figure 5. 13 Network monitoring



CHAPTER 6

6. Conclusion, Limitations and Future Works

6.1 Conclusion

This network security model provides a strong system for tracking and safeguarding network traffic and web server activity by integrating Suricata, Filebeat, Logstash, Kibana, and Elasticsearch. With the help of these free and open-source tools, one can quickly detect and address possible threats and abnormalities in real time. While Logstash analyses web application logs to track server performance and faults, Suricata watches network traffic for unusual behaviours.

Logs from Filebeat are rapidly forwarded to the main Elasticsearch database, where they are indexed for fast analysis and retrieval of data. As the layer for data visualization, Kibana provides an easy-to-use interface for analysing and visualizing data with the goal to spot trends, patterns, and security vulnerabilities. This model demonstrates how well-differentiated technologies can be combined to create a unified security and monitoring system that can adjust to changing application requirements and network settings.

The system's successful deployment highlights the need it is to integrate security components and conduct ongoing monitoring for the purpose to defend against changing cyberthreats. Organizations can improve their entire security posture by ensuring the integrity, availability, and confidentiality of their network and server resources by putting such a strong framework in place.

6.2 Limitations

There are some issues encountering during the setup of this system:

1. Integration challenges

It is difficult to integrate these technologies with other security solutions or the infrastructure that is already in place. Compatibility problems could occur, and more configuration and testing might be required to align formats for data or communication protocols between various systems.

2. DNS issue

There is delay propagating the DNS configuration and create connections between a domain name and the public IP address. Due to this delay, users can have frequent connectivity problems or find it difficult to use the online application, which could eventually cause outages and delays in service. This problem is particularly problematic in settings where ensuring high availability is critical. Also, the issue gets worse by frequent server restarts, which prolong the system's unavailability and responsiveness by starting the DNS reconfiguration process with each restart.

3. Complexity of setup and maintenance

The configuration and management of the system's numerous components requires a high degree of technical expertise to implement. Every tool has its own set of setups, conditions, and integration needs that is hard to demonstrate over time. Administrators may find this complexity to be quite complex, which will make it harder that misconfigurations will occur which harm system performance or security.

6.3 Future Works

This system can be expanded in the future and is scalable. Here are a few suggested changes to improve the system even more.

1. Firewall monitoring can be set up using the same method

The system is very robust and adaptable. Firewall monitoring can be set up in a manner like that of the network and web application monitoring systems. This security monitoring system would be substantially improved if firewall log analysis was included. Deeper analysis of firewall behaviour allows us to find important details like blocked traffic, attempted intrusions, and unusual patterns that could indicate security threats. The capacity to monitor firewall performance and security issues will also be greatly enhanced by building customized Kibana dashboards made just for firewall log data.

2. Enhanced automation and response with machine learning and AI integration

The system as entirety could be greatly improved by putting automated response systems into place. Security orchestration, automation, and response (SOAR) technologies reduce the time between attack detection and mitigation by automatically addressing threats in real-time [62]. Adding powerful machine learning and artificial intelligence (AI) methods can help the system identify more complex threats. Anomalous patterns in network traffic, for example, can be

found using anomaly detection algorithms, and complex attack scenarios can be found by connecting seemingly unconnected events using AI-driven relation engines.

3. Integration with cloud-based services

The necessity to integrate security measures with cloud-based services [63] like AWS Security Hub, Microsoft Azure Security Centre and Google Cloud Security Command Centre is expanding as more and more enterprises move their infrastructure to the cloud. This will enable the establishment of a hybrid security model that will efficiently monitor cloud and within the premises environments, guaranteeing thorough coverage of all infrastructure elements. This strategy not only improves security but also makes managing and supervising security processes easy.

References

1. Anderson, H. *et al.* (2010) 'No title', *Methods and Measures for Communicating Tactics, Techniques, and Procedures*,
2. Bejtlich, R. (2013) *The practice of network security monitoring: understanding incident detection and response*. No Starch Press.
3. Boiko, A. and Shendryk, V. (2017) 'System integration and security of information systems', *Procedia Computer Science*, 104, pp. 35–42.
4. Erdheim, S. (2013) 'Deployment and management with next-generation firewalls', *Network Security*, 2013(10), pp. 8–12.
5. Hussain, M. (2021) 'No title', *Availability, Integrity, and Confidentiality for Content Centric Network Internet Architectures*,
6. Kaleem, Y. (2022) 'No title', *Cyber Security Framework for Real-Time Malicious Network Traffic Detection and Prevention using SIEM and Deep Learning*,
7. Keromytis, A.D. and Prevelakis, V. (2007) 'Designing firewalls: A survey', *Network Security: Current Status and Future Directions*, , pp. 33–49.
8. PK, K. and Mbugua, J.K. (2014) 'Framework for examining firewall operation using network intrusion detection systems', *Kenya Studies*, , pp. 191.
9. Stewart, J.M. (2013) *Network security, firewalls and VPNs*. Jones & Bartlett Publishers.
10. A. A. Aryachandra, Y. F. Arif and S. N. Anggis (2016) 'Intrusion Detection System (IDS) server placement analysis in cloud computing', *2016 4th International Conference on Information and Communication Technology (ICoICT)*. Available at: 10.1109/ICoICT.2016.7571954.
11. A. Agarwal, S. Siddharth and P. Bansal (2016) 'Evolution of cloud computing and related security concerns', *2016 Symposium on Colossal Data Analysis and Networking (CDAN)*. Available at: 10.1109/CDAN.2016.7570920.
12. Al Shidhani, A. *et al.* (2016) 'A comparative analysis of open source network monitoring tools', *International Journal of Open Source Software and Processes (IJOSSP)*, 7(2), pp. 1–19.
13. Arifin, F.M., Mutiara, G.A. and Ismail, I. (2017) 'Implementation of management and network security using endian UTM firewall', *IJAIT (International Journal of Applied Information Technology)*, , pp. 43–51.
14. Berge, J. (2005) *Software for automation: architecture, integration, and security*. ISA.
15. Chakraborty, M. and Kundan, A.P. (2021) 'Grafana', in 'Grafana', *Monitoring cloud-native applications: Lead agile operations confidently using open source software*. Springer, pp. 187–240.
16. Cowley, J.A. and Greitzer, F.L. (2015) 'Organizational impacts to cybersecurity expertise development and maintenance', *Proceedings of the Human Factors and Ergonomics Society Annual Meeting*. SAGE Publications Sage CA: Los Angeles, CA
17. Dubrawsky, I. (2003) 'Firewall evolution-deep packet inspection', *Security Focus*, 29, pp.

21.

18. Eldeen, A.S. and Adam, A.M.E. 'DESIGNING OF WEB FILTERING POLICIES',
19. Fedorov, M. *et al.* (2017) 'No title', *Leveraging Splunk for Control System Monitoring and Management*,
20. Frank, J. (1994) 'Artificial intelligence and intrusion detection: Current and future directions', *Proceedings of the 17th national computer security conference*. Baltimore, MD
21. Freet, D. and Agrawal, R. (2016) 'Network security and next-generation firewalls', *Proceedings of International Conference on Technology Management (ICTM 2016)*.
22. Hamilton, J. *et al.* (2018) 'SCADA Statistics monitoring using the elastic stack (Elasticsearch, Logstash, Kibana)', *International Conference on Accelerator and Large Experimental Physics Control Systems*.
23. Hong, S. *et al.* (2015) 'Poisoning network visibility in software-defined networks: New attacks and countermeasures.', *Ndss*.
24. Ingham, K. and Forrest, S. (2002) 'A history and survey of network firewalls', *University of New Mexico, Tech.Rep*,
25. J. Hurley, A. Munoz and S. Sezer (2012) 'ITACA: Flexible, scalable network analysis', *2012 IEEE International Conference on Communications (ICC)*. Available at: 10.1109/ICC.2012.6363995.
26. J. R. Yost (2016) 'The March of IDES: Early History of Intrusion-Detection Expert Systems', *IEEE Annals of the History of Computing*, 38(4), pp. 42–54. Available at: <https://doi.org/10.1109/MAHC.2015.41>
27. K. Neupane, R. Haddad and L. Chen (2018) 'Next Generation Firewall for Network Security: A Survey', *SoutheastCon 2018*. Available at: 10.1109/SECON.2018.8478973.
28. M. Cheminod *et al.* (2022) 'Open-source firewalls for industrial applications: a laboratory study of Linux IPFire behavior', *IECON 2022 – 48th Annual Conference of the IEEE Industrial Electronics Society*. Available at: 10.1109/IECON49645.2022.9969064.
29. M. Cinque, D. Cotroneo and A. Pecchia (2018) 'Challenges and Directions in Security Information and Event Management (SIEM)', *2018 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW)*. Available at: 10.1109/ISSREW.2018.00-24.
30. M. Fuentes-García, J. Camacho and G. Maciá-Fernández (2021) 'Present and Future of Network Security Monitoring', *IEEE Access*, 9, pp. 112744–112760. Available at: <https://doi.org/10.1109/ACCESS.2021.3067106>
31. M. G. Gouda and A. X. Liu (2005) 'A model of stateful firewalls and its properties', *2005 International Conference on Dependable Systems and Networks (DSN'05)*. Available at: 10.1109/DSN.2005.9.
32. M. Waheed and M. Cheng (2017) 'A system for real-time monitoring of cybersecurity events on aircraft', *2017 IEEE/AIAA 36th Digital Avionics Systems Conference (DASC)*. Available at: 10.1109/DASC.2017.8102120.

33. McWeeney, K. (2015) 'An implementation and evaluation of PCI DSS 3.0 for E-Commerce in a testing environment',
34. Mito, W. and Matsunaga, M. (2016) 'Cloud/crowd sensing system for annotating users perception', *Journal of Robotics and Mechatronics*, 28(1), pp. 61–68.
35. Muhammad, A.R., Sukarno, P. and Wardana, A.A. (2023) 'Integrated Security Information and Event Management (SIEM) with Intrusion Detection System (IDS) for Live Analysis based on Machine Learning', *Procedia Computer Science*, 217, pp. 1406–1415. Available at: <https://doi.org/10.1016/j.procs.2022.12.339>
36. Naing, T. and Oo, A.N. 'No title', *Design and Implementation of A Secure Network Connection of the University*,
37. O. Adeyinka (2008) 'Internet Attack Methods and Internet Security Technology', *2008 Second Asia International Conference on Modelling & Simulation (AMS)*. Available at: 10.1109/AMS.2008.68.
38. Olups, R. (2010) *Zabbix 1.8 network monitoring*. Packt Publishing Ltd.
39. P. SenthilKumar and M. Muthukumar (2018) 'A Study on Firewall System, Scheduling and Routing using pfsense Scheme', *2018 International Conference on Intelligent Computing and Communication for Smart World (I2C2SW)*. Available at: 10.1109/I2C2SW45816.2018.8997167.
40. Renita, J. and Elizabeth, N.E. (2017) 'Network's server monitoring and analysis using Nagios', *2017 International Conference on Wireless Communications, Signal Processing and Networking (WiSPNET)*. IEEE
41. Rustam, M. 'The Development of Media in the 20th century',
42. Sampaio, D. and Bernardino, J. (2017) 'Evaluation of firewall open source software', *International Conference on Web Information Systems and Technologies*. SCITEPRESS
43. Sarker, I.H. (2023) 'Machine learning for intelligent data analysis and automation in cybersecurity: current and future prospects', *Annals of Data Science*, 10(6), pp. 1473–1498.
44. Shinde, S., Bangar, A. and Tawde, M. 'Comparative Study and Analysis of Ids Implementation In Cloud Computing Environment', *IOSR Journal of Computer Science (IOSR-JCE) E ISSN*, , pp. 2278–0661.
45. Stubbig, M. (2019) *Practical OPNsense: Enterprise firewalls build on open source*. BoD–Books on Demand.
46. Sulaman, S.M. (2011) 'An Analysis and Comparison of The Security Features of Firewalls and IDSs',
47. Tawalbeh, L. *et al.* (2020) 'IoT Privacy and security: Challenges and solutions', *Applied Sciences*, 10(12), pp. 4102.
48. Thakur, A. (2015) 'Open source firewall implementation: replacing traditional firewall with open source',
49. Theodoropoulos, T. *et al.* (2023) 'Security in Cloud-Native Services: A Survey', *Journal of Cybersecurity and Privacy*, 3(4), pp. 758–793.

50. Toumi, H. *et al.* (2019) 'Implementing Hy-IDS, mobiles agents and virtual firewall to enhance the security in IaaS Cloud', *Procedia Computer Science*, 160, pp. 819–824.
51. Turnbull, J. (2018) *Monitoring with Prometheus*. Turnbull Press.
52. W. Li, W. Meng and L. F. Kwok (2022) 'Surveying Trust-Based Collaborative Intrusion Detection: State-of-the-Art, Challenges and Future Directions', *IEEE Communications Surveys & Tutorials*, 24(1), pp. 280–305. Available at: <https://doi.org/10.1109/COMST.2021.3139052>
53. W. Yassin *et al.* (2012) 'A Cloud-based Intrusion Detection Service framework', *Proceedings Title: 2012 International Conference on Cyber Security, Cyber Warfare and Digital Forensic (CyberSec)*. Available at: 10.1109/CyberSec.2012.6246098.
54. Wilson, J. (2005) 'The future of the firewall', *Business Communications Review*, 5, pp. 28–32.
55. Xuan, S. *et al.* (2016) 'Performance evaluation model for application layer firewalls', *PLoS One*, 11(11), pp. e0167280.
56. Y. -X. Ding, M. Xiao and Ai-Wu Liu (2009) 'Research and implementation on snort-based hybrid intrusion detection system', *2009 International Conference on Machine Learning and Cybernetics*. Available at: 10.1109/ICMLC.2009.5212282.
57. Youssef, B.C., Nada, M. and Boubker, R. (2019) 'Behavioural analysis approach for IDS based on attack pattern and risk assessment in cloud computing', *International Journal of Information and Computer Security*, 11(4-5), pp. 315–331.
58. Kouka, A. (2015) *Ubuntu Server Essentials*. Packt Publishing Ltd.
59. LaCroix, J. (2016) *Mastering Ubuntu Server*. Packt Publishing Ltd.
60. Waleed, A., Jamali, A.F. and Masood, A. (2022) 'Which open-source ids? Snort, suricata or zeek', *Computer Networks*, 213, pp. 109116.
61. Zámotný, T. (2021) 'Telemetry Data Collection, Analysis and Representation',
62. Bartwal, U. *et al.* (2022) 'Security orchestration, automation, and response engine for deployment of behavioural honeypots', *2022 IEEE Conference on Dependable and Secure Computing (DSC)*. IEEE
63. ȘTEFAN, A., OVREIU, E. and CIUC, M. (2024) 'Comparative analysis of web-based machine learning models.', *Romanian Journal of Information Technology & Automatic Control/Revista Română De Informatică Și Automatică*, 34(2)
64. 'ubuntu', Available at: <https://ubuntu.com/>
65. *VirtualBox*. Available at: <https://www.virtualbox.org/>
66. *pfsense*. Available at: <https://www.pfsense.org/>

Appendices

Appendix A: Commands used for Ubuntu server

Update package sudo apt

update Find ip address of

the server

```
ip a
```

ip addr

Appendix B: Installation of Apache2

Install Apache2 sudo apt install apache2 -y

Start Apache2 sudo systemctl start apache2

Enable Apache2 sudo systemctl enable

apache2 Status of Apache2 sudo

systemctl status apache2 Restart Apache2

sudo systemctl restart apache2 Check

errors in Apache2 sudo journalctl -xe |

grep apache2

Appendix C: Commands, HTML and styles used of web application

Make directory

```
mkdir webapp
```

```
mkdir logs
```

```
mkdir css
```


Create file and edit

```
sudo nano index.html
```

```
sudo nano about.html
```

```
sudo nano contact.html
```

```
sudo nano styles.css Website
```

codes:

File: index.html

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Home - My Big Web Application</title>

  <link rel="stylesheet" href="css/styles.css">

</head>

<body>

  <header>

    <h1>Welcome to My Big Web Application</h1>

    <nav>

      <ul>

        <li><a href="index.html">Home</a></li>

        <li><a href="about.html">About</a></li>

        <li><a href="contact.html">Contact</a></li>
```

```
        </ul>

    </nav>

</header>

<main>

    <p>This is the home page of the web application.</p>

</main>

<footer>

    <p>&copy; 2024 My Big Web Application</p>

</footer>

</body>

</html>
```

File: about.html

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>About - My Big Web Application</title>

    <link rel="stylesheet" href="css/styles.css">

</head>

<body>

    <header>
```

```
<h1>About My Big Web Application</h1>

<nav>

  <ul>

    <li><a href="index.html">Home</a></li>

    <li><a href="about.html">About</a></li>

    <li><a href="contact.html">Contact</a></li>

  </ul>

</nav>

</header>

<main>

  <p>This page contains information about the web application.</p>

</main>

<footer>

  <p>&copy; 2024 My Big Web Application</p>

</footer>

</body>

</html>
```

File: contact.html

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Contact - My Big Web Application</title>

<link rel="stylesheet" href="css/styles.css">

</head>

<body>

  <header>

    <h1>Contact Us</h1>

    <nav>

      <ul>

        <li><a href="index.html">Home</a></li>

        <li><a href="about.html">About</a></li>

        <li><a href="contact.html">Contact</a></li>

      </ul>

    </nav>

  </header>

  <main>

    <p>This page contains contact information.</p>

  </main>

  <footer>

    <p>&copy; 2024 My Big Web Application</p>

  </footer>

</body>

</html>
```

File: styles.css

```
body { font-family: Arial,
sans-serif; margin: 0;
padding: 0; background-
color: #f4f4f4;
} header { background-
color: #333;
color: #fff;
padding: 1em 0; text-
align: center; } nav ul {
list-style: none;
padding: 0; } nav ul li {
display: inline;
margin: 0 1em; } nav ul
li a { color: #fff;
text-decoration: none;
}
main {
padding: 2em;
} footer { text-align:
center; padding: 1em 0;
background-color: #333;
```

```
        color: #fff;  
    }  
}
```

Open access.log

```
cat accesss.log
```

Open error.log

```
cat error.log
```

Appendix D: Deploying website and configuration

Disable default site `sudo a2dissite 000-`

`default.conf` Enable new site `sudo`

`a2ensite test.project.final.conf` Reload new

change `sudo systemctl reload`

apache2

website configuration file : `test.project.final.conf`

```
<VirtualHost *:80>
```

```
    ServerAdmin webmaster@localhost
```

```
    ServerName theanugrah.com
```

```
    ServerAlias www.theanugrah.com
```

```
    ErrorLog /var/www/webapp/logs/error.log
```

```
    CustomLog /var/www/webapp/logs/access.log combined
```

```
<Directory /var/www/html/webapp>
```

```
    Options -Indexes +FollowSymLinks
```

```
        AllowOverride All

        Require all granted

    </Directory>

    ErrorLog ${APACHE_LOG_DIR}/error.log

    CustomLog ${APACHE_LOG_DIR}/access.log combined

</VirtualHost>
```

Appendix E: UFW in ubuntu server

Allow SSH sudo ufw allow

ssh Allow HTTP sudo

ufw allow http Allow

HTTPS sudo ufw

allow https Enable UFW

sudo ufw enable

Disable UFW

sudo ufw disable UFW

status sudo ufw status

Appendix F: Installing Java

Install Java sudo apt install open-jdk

jdk -y Check java version

sudo java -version

Appendix G: Installing Elasticsearch

Install the GPG key

```
sudo wget -qO - https://artifacts.elastic.co/GPG-KEY-elasticsearch | sudo apt-key add
```

-

Add the repository

```
sudo echo "deb https://artifacts.elastic.co/packages/7.x/apt stable main" | sudo tee  
/etc/apt/sources.list.d/elasticsearch-7.x.list
```

Install Elasticsearch `sudo apt install`

`elasticsearch -y` Enable Elasticsearch

`sudo systemctl enable elasticsearch` Start

Elasticsearch `sudo systemctl start`

`elasticsearch` Edit Elasticsearch

configuration file

```
sudo nano elasticsearch.yml
```

Elasticsearch configuration file: `elasticsearch.yml`

`path.data: /var/lib/elasticsearch` `path.data:`

`/var/log/elasticsearch` `network.host:`

`192.168.1.102` `http.port: 9200` `discovery.type:`

`single-node` Edit Elasticsearch `jvm.options`

```
sudo nano jvm.options
```

Elasticsearch options file: `jvm.options`

```
-xmx512m
```

```
-xmx512m
```


Restart Elasticsearch `sudo systemctl restart`

elasticsearch Elasticsearch status `sudo systemctl`

status elasticsearch Check errors in Elasticsearch

`sudo journalctl -u elasticsearch sudo less`

`/var/log/elasticsearch/elasticsearch.log`

Appendix H: Installing Logstash

Install Logstash `sudo apt`

`install logstash -y` Edit Logstash

configuration file

`sudo nano logstash.conf`

Logstash configuration file: `logstash.conf`

```
input {  
  
  file {  
  
    path =>  
  
    "/var/www/webapp/logs/access.log"  
  
    start_position => "beginning"    sincedb_path  
=> "/dev/null"  
  
  }  
  
  file {  
  
    path => "/var/www/webapp/logs/error.log"  
  
    start_position => "beginning"  
  
    sincedb_path => "/dev/null"
```

```

    } }

filter {

  grok {

    match => { "message" => "%{COMMONAPACHELOG}" }

  }

  date {

    match => [ "timestamp" , "dd/MMM/yyyy:HH:mm:ss Z" ]

  } } output { elasticsearch { hosts =>

["http://192.168.1.102:9200"] index

=> "webapp-logs-%{+YYY.MM.dd}"

}

stdout { codec => rubydebug }

}

```

Enable Logstash `sudo systemctl`

`enable logstash`

Start logstash `sudo systemctl start`

logstash Restart Logstash `sudo`

`systemctl restart logstash Logstash`

status `sudo systemctl status logstash`

Check errors is Logstash `sudo`

`journalctl -u logstash`

```
sudo less /var/log/logstash/logstash.log
```

Appendix I: Installing Kibana

```
Install Kibana sudo apt install
```

```
kibana -y Edit Kibana
```

```
configuration file
```

```
sudo nano kibana.yml
```

```
Kibana configuration file: kibana.yml
```

```
server.port: 5601
```

```
server.host: "192.168.1.102" elasticsearch.hosts:
```

```
["http://192.168.1.102:9200"]
```

```
Check Filebeat configuration sudo filebeat test
```

```
config -c /etc/filebeat/filebeat.yml Enable Kibana sudo
```

```
systemctl enable kibana Start Kibana sudo
```

```
systemctl start kibana Restart Kibana sudo
```

```
systemctl restart kibana
```

```
Check errors in Kibana sudo journalctl -u
```

```
kibana sudo less
```

```
/var/log/kibana/kibana.log
```

Appendix J: Creating Index pattern in ELK

Step 1: Go to Management in Kibana in ELK web interface

Step 2: Select index patterns

Step 3: Then, create index pattern and search for indices for the pattern created

Step 4: Include all indices starting with "filebeat-", enter "filebeat-*" or "webapp-logs-", enter "webapp-logs-*".

Step 5: Click Next Step to proceed and select time field "@timestamp" Step

6: Finally, create Index pattern.

The below figure shows the interface for creating the index pattern.

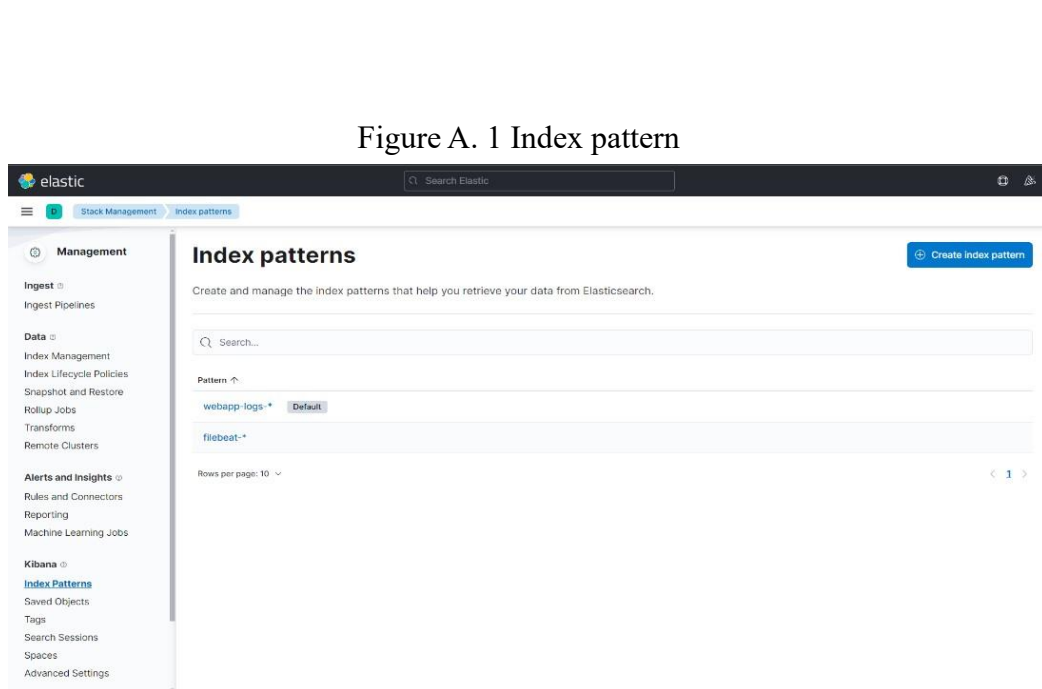


Figure A. 1 Index pattern

Appendix K: Installing Suricata

Add the repository `sudo add-apt-repository`

`ppa:oisf/suricata-stable` Install Suricata `sudo apt-`

`get install suricata -y` Edit Suricata configuration file

`sudo nano suricata.yaml`

Suricata configuration file: `suricata.yml`

`af-packet:`

`- interface: enp0s3`

Enable Suricata `sudo systemctl`

`enable suricata`

Start Suricata `sudo systemctl start`

`suricata`

Restart Suricata

`sudo systemctl restart suricata`

Suricata status `sudo systemctl`

`status suricata` Check errors in

Suricata `sudo journalctl -u`

`suricata`

`sudo less /var/log/suricata/suricata.log` Open

`eve.json`

`sudo nano eve.json`

Appendix L: Installing Filebeat

Install the GPG key `wget -qO - https://artifacts.elastic.co/GPG-KEY-elasticsearch | sudo apt-key add -`

Add the repository

`echo "deb https://artifacts.elastic.co/packages/7.x/apt stable main" | sudo tee -a /etc/apt/sources.list.d/elastic-7.x.list` Install Filebeat `sudo apt-get install filebeat -y` Edit

Filebeat configuration file

`sudo nano filebeat.yml`

Filebeat configuration file: `filebeat.yml`

```

filebeat.inputs: -

type: log

enabled: true

paths:

  - /var/log/suricata/eve.json

setup.kibana: host:

"http:192.168.1.102:5601"

setup.ilm.enabled: auto

setup.ilm.overwrite: true

output.elasticsearch:

  hosts: ["http:192.168.1.102:9200"]

```

Test configuration file of filebeat `sudo filebeat test`

`config -c /etc/filebeat/filebeat.yml` Check error in filebeat

`sudo journalctl -u filebeat` `sudo less`

`/var/log/filebeat/filebeat.log`

Load Filebeat index into Elasticsearch `sudo filebeat setup --index-management -E`

`output.logstash.enabled=false -E 'output.elasticsearch.hosts=["http:192.168.1.102:9200"]'`

Enable Filebeat `sudo systemctl enable filebeat`

Start Filebeat `sudo systemctl start`

filebeat Restart Filebeat `sudo`

`systemctl restart filebeat` Filebeat

status `sudo systemctl status filebeat`

Filebeat.log

```
sudo nano filebeat.log
```